

C++ 题库

按 `references/` 14 个主题分章；每章先题目、后参考答案与知识点总结。
取材自语法汇总与考试题库，适合系统复习。建议先修 [C 语言](#)。

使用说明

1. 建议按章顺序练习；也可按薄弱主题跳章。
2. 先独立完成「一、题目」，再对照「二、参考答案与解析」。
3. 「三、本章知识点总结」用于考前速记。
4. 读程序题默认 C++11/14/17、常见 64 位环境。
5. 全书约 343 题（第 1~7 章各 25 题，第 8~14 章各 24 题），分 14 章。

目录

1. [程序基础](#)
2. [数据类型与常量](#)
3. [运算符](#)
4. [流程控制](#)
5. [函数与重载](#)
6. [引用与指针](#)
7. [数组与 string](#)
8. [类与对象](#)
9. [继承与多态](#)
10. [模板](#)
11. [STL 容器](#)
12. [STL 算法](#)
13. [文件流](#)
14. [智能指针与常用头文件](#)

第 1 章 · 程序基础

本章知识点速览

- 编译四阶段：预处理 → 编译 → 汇编 → 链接；常用 `g++ -std=c++17`
- `main` 返回 `int`，`return 0` 表示正常结束
- `#include <iostream>`；`std::cout` / `std::cin` 在 `std` 命名空间
- 命名空间：`namespace`、`::`、`using`；头文件中避免 `using namespace std;`
- 源文件 `.cpp`，头文件 `.h` / `.hpp`；`#pragma once` 或 include guard
- C++ 相对 C 的额外关键字：`namespace`、`bool`、`nullptr`、`class`、`new` 等

一、题目

选择题

1. 下列关于 C++ 编译流程的说法, 正确的是:
 - A. 预处理直接生成可执行文件
 - B. 链接阶段处理 `#include` 与 `#define`
 - C. 编译阶段将 C++ 代码翻译为汇编代码
 - D. 汇编阶段合并目标文件与库
2. C++ 程序的入口函数通常是:
 - A. `start`
 - B. `main`
 - C. `init`
 - D. `cout`
3. 下列哪个是合法的标识符?
 - A. `2count`
 - B. `my-var`
 - C. `_total`
 - D. `int`
4. 关于 `.cpp` 与 `.h` / `.hpp` 文件, 正确的是:
 - A. `.hpp` 中通常写全部函数实现, 不能放声明
 - B. `.cpp` 中通常只写宏定义供他人包含
 - C. `.cpp` 存放实现, 头文件存放声明, 由 `#include` 引入
 - D. 两者完全等价, 可随意互换
5. 下列关于注释的说法, 错误的是:
 - A. `//` 是 C++ 原生支持的单行注释
 - B. `/* ... */` 可以嵌套使用
 - C. `/* ... */` 可用于多行注释
 - D. 注释内容不参与编译后的可执行逻辑
6. 下列哪个是 C++ 关键字 (相对 C 的扩展) ?
 - A. `main`
 - B. `printf`
 - C. `nullptr`
 - D. `Hello`
7. `g++ -c hello.cpp` 的作用是:
 - A. 编译并链接生成可执行文件
 - B. 只预处理不编译
 - C. 只编译不链接, 生成目标文件
 - D. 直接运行程序
8. 预处理阶段的常见产物扩展名是:
 - A. `.s`
 - B. `.i`
 - C. `.o`
 - D. `.exe`
9. 关于命名空间, 正确的是:
 - A. `std::cout` 中的 `::` 是流插入运算符
 - B. `using namespace std;` 适合写在头文件全局区域
 - C. 可用 `using std::cout;` 只引入单个名称
 - D. 标准库符号都不在任何命名空间中
10. `#include <iostream>` 中尖括号表示:
 - A. 优先在当前目录查找头文件
 - B. 在系统/标准头文件搜索路径中查找

- C. 只用于宏定义
D. 与 `#include "iostream"` 完全等价且无差别
11. 下列关于 C++ 特点的说法，错误的是：
- A. 兼容绝大部分 C 语法
B. 支持面向对象与泛型等范式
C. 标准库内容都在 `std` 命名空间
D. 运行时自动回收全部动态内存，程序员无需关心释放
12. `g++ -std=c++17 hello.cpp -o hello` 中，`-o hello` 的作用是：
- A. 开启 C++17 标准
B. 指定输出可执行文件名
C. 只编译不链接
D. 开启全部警告

填空题

13. 编译四阶段依次是：预处理、__、汇编、__。
14. C++ 源文件扩展名通常是 __，头文件扩展名常见为 __ 或 __.hpp。
15. C++ 标准 I/O 头文件是 __；标准输出对象是 __。
16. 访问命名空间中名称需使用作用域解析运算符 __。
17. `int main()` 中，`return 0;` 一般表示程序 __ 结束。
18. 防止头文件被重复包含，可用 `#pragma _____` 或 `#ifndef / #define / #endif`。

读程序写结果

19. 写出程序输出：

```
#include <iostream>
int main() {
    std::cout << "Hello, C++!\n";
    return 0;
}
```

20. 写出程序输出：

```
#include <iostream>
using std::cout;
int main() {
    cout << "A" << "B\n";
    return 0;
}
```

21. 写出程序输出：

```
#include <iostream>
int main() {
    /* std::cout << "A"; */
    // std::cout << "B";
    std::cout << "C";
    return 0;
}
```

22. 写出程序输出：

```
#include <iostream>
namespace Math {
    int add(int a, int b) { return a + b; }
}
int main() {
    std::cout << Math::add(2, 3) << "\n";
    return 0;
}
```

简答或编程

23. 简述预处理、编译、汇编、链接四个阶段各自的主要工作，并写出各阶段常见产物扩展名。
24. 说明 `using std::cout;` 与 `using namespace std;` 的区别，并说明为何头文件中应避免后者。
25. 写出一个最小可运行 C++ 程序，要求输出一行 `Hi`（含换行），使用 `iostream`。

二、参考答案与解析

1. 答案：C

解析：预处理处理 `#` 指令；编译生成汇编；汇编生成目标文件；链接生成可执行文件。

2. 答案：B

解析：C++ 程序从 `main` 开始执行，且通常返回 `int`。

3. 答案：C

解析：`_total` 合法；不能以数字开头，不能含 `-`，不能是关键字。

4. 答案：C

解析：约定是实现与声明分离：`.cpp` 实现，头文件声明。

5. 答案：B

解析：`/* */` 注释不能嵌套。

6. 答案：C

解析：`nullptr` 是 C++11 关键字；`main`、`printf` 不是关键字。

7. 答案：C

解析：`-c` 表示只编译到目标文件（`.o` / `.obj`），不链接。

8. 答案：B

解析：预处理产物常为 `.i`；编译 `.s`；汇编 `.o`；链接可执行文件。

9. 答案：C

解析：`using` 声明可只引入单个名称；`::` 是作用域解析；头文件避免 `using namespace std;`。

10. 答案：B

解析：尖括号通常搜系统路径；双引号通常先搜当前目录。

11. 答案：D

解析：C++ 无自动垃圾回收；`new` / `delete` 或智能指针需程序员管理资源。

12. 答案：B

解析：`-o` 指定输出文件名；`-std=c++17` 指定语言标准。

13. 答案：编译；链接

解析：四阶段固定顺序：预处理→编译→汇编→链接。

14. 答案：.cpp; .h (或 .hpp)

解析：也可见 `.cc` / `.cxx`；头文件常见 `.h` / `.hpp`。

15. 答案：`<iostream>`；`std::cout`

解析：标准输入为 `std::cin`，标准错误为 `std::cerr`。

16. 答案：`::`

解析：如 `std::string`、`Math::add`。

17. 答案：正常（成功）

解析：返回 0 通常表示成功，非 0 表示异常。

18. 答案：once

解析：`#pragma once` 与传统 `include guard` 均可防止重复包含。

19. 答案：

```
Hello, C++!
```

解析：标准最小程序输出，``\\n`` 换行。

20. 答案：

```
AB
```

解析：``using std::cout;`` 后可直接写 ``cout``，连续插入输出。

21. 答案：

```
C
```

解析：前两行输出被注释掉，只打印 ``C``。

22. 答案：

```
5
```

解析：通过 `Math::add` 调用命名空间中函数，返回 2+3。

23. 答案要点：

预处理：处理 `#include` / `#define` 等，常见产物 `.i`；编译：C++→汇编，`.s`；汇编：汇编→机器码目标文件，`.o` / `.obj`；链接：合并目标文件与库，生成可执行文件。

24. 答案要点：

`using std::cout;` 只引入一个名称；`using namespace std;` 引入整个 `std`。头文件中使用后者会污染包含该头文件的所有翻译单元，易产生名字冲突。

25. 参考代码：

```
#include <iostream>
int main() {
    std::cout << "Hi\n";
    return 0;
}
```

三、本章知识点总结

要点	速记
编译流程	预→编→汇→链；常用 <code>g++ -std=c++17</code>
入口	<code>main</code> 返回 <code>int</code> ， <code>return 0</code> 成功
I/O	<code>#include <iostream></code> ， <code>std::cout</code> / <code>cin</code>
命名空间	<code>namespace</code> 、 <code>::</code> 、慎用 <code>using namespace</code>
文件	<code>.cpp</code> 实现 / <code>.h</code> <code>.hpp</code> 声明
注释	<code>//</code> 与 <code>/* */</code> ，后者不可嵌套

第 2 章 · 数据类型与常量

本章知识点速览

- 基本类型含 `bool` / `char` / `int` / `float` / `double`；`true` / `false`
- 引用 `T&` 是别名，必须初始化；常引用 `const T&` 常作参数
- `auto` 由初始化式推导类型；`decltype` 获取表达式类型
- `const` 只读；`constexpr` 编译期常量（C++11）
- 初始化：拷贝 / `()` / `{}` 列表初始化；`iostream` 用 `<<` / `>>`
- `std::boolalpha` 使 `bool` 输出为 `true` / `false` 文字

一、题目

选择题

1. 下列关于 `bool` 的说法，正确的是：
 - A. C++ 中没有布尔类型，只能用 `int`
 - B. `true` 与 `false` 是 `bool` 的两个字面值
 - C. `bool` 变量只能赋值为 0 或 1 的整数字面量
 - D. `bool` 不能用于 `if` 条件
2. `int x = 3.9;` 执行后 `x` 的值是：
 - A. 3.9
 - B. 4
 - C. 3
 - D. 不确定
3. 关于引用，正确的是：
 - A. `int& r;` 合法，可稍后绑定
 - B. 引用是别名，必须初始化且一般不可再绑到别的对象
 - C. 引用与指针完全等价
 - D. `const int&` 可以修改所绑定对象
4. `auto x = 42;` 中 `x` 的类型通常是：
 - A. `double`
 - B. `int`
 - C. `long`
 - D. `bool`
5. 下列哪一项是列表初始化 (C++11) ?
 - A. `int x = 10;`
 - B. `int x(10);`
 - C. `int x{10};`
 - D. `int x;`
6. 关于 `const` 与 `constexpr`，错误的是：
 - A. `const` 表示只读
 - B. `constexpr` 强调编译期可求值的常量
 - C. `constexpr` 变量必须能在编译期确定
 - D. `const` 与 `constexpr` 含义完全相同，可随意互换
7. `std::cout << std::boolalpha << true;` 的输出通常是：
 - A. 1
 - B. `true`
 - C. `True`
 - D. 无输出
8. 下列关于 `iostream` 的说法，正确的是：
 - A. `std::endl` 只换行，不刷新缓冲区
 - B. `std::cin >> x` 从标准输入提取
 - C. `std::cerr` 是标准输入
 - D. 必须用 `printf` 才能输出整数
9. `const int& r = 10;` 的说法，正确的是：
 - A. 非法，引用不能绑定临时量
 - B. 合法，常引用可绑定临时量 (延长生命周期)
 - C. 可以通过 `r` 修改值为 20
 - D. `r` 的类型是 `int*`

10. 下列初始化中, `auto` 推导为 `double` 的是:

- A. `auto a = 1;`
- B. `auto b = 1.0;`
- C. `auto c = true;`
- D. `auto d = 'A';`

11. 关于 `decltype`, 正确的是:

- A. `decltype` 用于运行时强制类型转换
- B. `decltype(expr)` 获取表达式的类型
- C. `decltype` 只能用于函数返回值
- D. `decltype` 与 `auto` 完全相同

12. `float f = 1.5f;` 中后缀 `f` 的作用是:

- A. 表示十六进制
- B. 表示该字面量为 `float` 类型
- C. 表示无符号
- D. 无实际作用

填空题

13. C++ 布尔类型关键字是 __, 其字面值为 __ 与 __。

14. 引用声明形如 `int& r = x;`, 其中 `r` 是 `x` 的 __。

15. `auto s = "hi";` 在未特别指定时, `s` 类型通常是指向字符常量的 __ (不是 `std::string`)。

16. 标准输入对象是 __, 标准输出对象是 __。

17. 用常引用避免大对象拷贝的参数写法示例: `void f(const std::string& _____);`。

18. `constexpr int N = 256;` 表示 `N` 是 __ 期常量。

读程序写结果

19. 写出程序输出:

```
#include <iostream>
int main() {
    bool ok = true;
    std::cout << ok << " ";
    std::cout << std::boolalpha << ok << "\n";
    return 0;
}
```

20. 写出程序输出:

```
#include <iostream>
int main() {
    int x = 10;
    int& r = x;
    r = 20;
    std::cout << x << " " << r << "\n";
    return 0;
}
```

21. 写出程序输出:


```
#include <iostream>
int main() {
    auto a = 3;
    auto b = 2.5;
    std::cout << a + b << "\n";
    return 0;
}
```

22. 写出程序输出：

```
#include <iostream>
void add_one(int& n) { n = n + 1; }
int main() {
    int x = 5;
    add_one(x);
    std::cout << x << "\n";
    return 0;
}
```

简答或编程

23. 简述引用与指针至少三点区别（初始化、空值、重新绑定等）。
24. 说明为什么函数参数常用 `const T&` 传递较大对象（如 `std::string`）。
25. 写一段代码：定义 `int a = 1;`，用引用 `r` 把 `a` 改为 100，再用 `std::cout` 输出 `a`。

二、参考答案与解析

1. 答案：B

解析：C++ 有独立的 `bool` 类型及 `true` / `false`。

2. 答案：C

解析：浮点赋给整型会截断小数部分，得到 3。

3. 答案：B

解析：引用必须初始化，是对象的别名。

4. 答案：B

解析：整数字面量 `42` 的类型是 `int`。

5. 答案：C

解析：花括号 `{}` 是 C++11 统一列表初始化。

6. 答案：D

解析：`constexpr` 更强调编译期常量；二者并不完全等价。

7. 答案：B

解析：`boolalpha` 使布尔以文字形式输出。

8. 答案: B

解析: `cin` 提取输入; `endl` 换行并刷新; `cerr` 是标准错误。

9. 答案: B

解析: 常引用可绑定临时量; 非常量引用一般不能。

10. 答案: B

解析: `1.0` 是 `double` 字面量。

11. 答案: B

解析: `decltype` 在编译期得到表达式类型。

12. 答案: B

解析: 无后缀浮点字面量默认是 `double`; `f`/`F` 表示 `float`。

13. 答案: `bool`; `true`; `false`

解析: 布尔类型及相关字面值。

14. 答案: 别名

解析: 对 `r` 的读写即对 `x` 的读写。

15. 答案: 指针 (`const char*`)

解析: 字符串字面量类型是字符数组, `auto` 会衰减为指针; 要用 `std::string` 需显式写类型或 `"s"` 等。

16. 答案: `std::cin`; `std::cout`

解析: 均在 `<iostream>` 的 `std` 中。

17. 答案: `s` (参数名, 如 `s/str/name` 均可)

解析: 形参名可自定, 关键是 `const std::string&`。

18. 答案: 编译

解析: `constexpr` 要求编译期可求值。

19. 答案:

```
1 true
```

解析: 默认 ``bool`` 输出 `0/1`; 开启 ``boolalpha`` 后输出文字。

20. 答案:

```
20 20
```

解析: 通过引用修改即修改原变量。

21. 答案:

5.5

解析：`int` 与 `double` 相加提升为 `double`。

22. 答案：

6

解析：引用传参可修改实参。

23. 答案要点：

引用必须初始化，指针可以稍后赋值；引用一般不能为空（非常量引用），指针可为 `nullptr`；引用通常不可重新绑定，指针可指向别处；引用使用无需解引用运算符。

24. 答案要点：

避免拷贝开销；`const` 保证函数内不修改实参，接口更安全清晰。

25. 参考代码：

```
#include <iostream>
int main() {
    int a = 1;
    int& r = a;
    r = 100;
    std::cout << a << "\n";
    return 0;
}
```

三、本章知识点总结

要点	速记
bool	<code>true</code> / <code>false</code> ； <code>boolalpha</code> 输出文字
引用	别名，必须初始化； <code>const T&</code> 常用
auto	由初始化推导类型
常量	<code>const</code> 只读； <code>constexpr</code> 编译期
初始化	<code>=</code> / <code>()</code> / <code>{}</code>
I/O	<code>cin >></code> / <code>cout <<</code> / <code>endl</code>

第 3 章 · 运算符

本章知识点速览

- 算术: `+` `-` `*` `/` `%`; 整数除法截断; 关系/逻辑结果类型为 `bool`
- 赋值与复合赋值、自增自减 `++` / `--`
- C++ 扩展: `<<` / `>>` (流)、`::` (作用域)、`new` / `delete`、`nullptr`
- 位运算: `&` `|` `^` `~` `<<` `>>` (注意与流插入/提取同形异义, 依操作数而定)
- 优先级与结合性大体同 C; 条件运算符 `?:`

一、题目

选择题

1. 表达式 `7 / 2` (两边均为 `int`) 的结果是:
A. 3.5
B. 3
C. 4
D. 3.0
2. 表达式 `7 % 2` 的结果是:
A. 3
B. 1
C. 0
D. 3.5
3. C++ 中 `3 < 5` 的结果类型通常是:
A. `int`
B. `bool`
C. `char`
D. `void`
4. 关于短路求值, 正确的是:
A. `&&` 左侧为假时仍求右侧
B. `||` 左侧为真时不再求右侧
C. `!` 也会短路
D. 关系运算符会短路
5. `std::cout << x` 中的 `<<` 在此语境下表示:
A. 左移运算符
B. 流插入 (输出)
C. 作用域解析
D. 按位与
6. 下列哪个用于空指针 (C++11 起推荐)?
A. `NULL` 只能, `nullptr` 非法
B. `nullptr`
C. `0L` 专用于指针
D. `void`
7. `int* p = new int(5);` 之后应如何释放?
A. `free(p);`
B. `delete p;`
C. `delete[] p;`
D. 无需释放
8. 关于 `new int[10]` 与释放, 正确的是:
A. 用 `delete p;`

- B. 用 `delete[] p;`
C. 用 `free(p);`
D. 自动释放，无需操作
9. `std::cin >> x` 中的 `>>` 表示：
A. 右移
B. 流提取（输入）
C. 大于大于
D. 命名空间成员访问
10. 表达式 `a = b = 3;` 的结合性是：
A. 从左到右
B. 从右到左
C. 无结合性
D. 取决于类型
11. `::` 运算符的主要用途是：
A. 整数除法
B. 作用域解析
C. 成员指针解引用
D. 强制类型转换
12. 下列关于自增的说法，错误的是：
A. `++i` 为前置自增
B. `i++` 为后置自增
C. 前置与后置在单独语句中对 `i` 的最终效果通常相同
D. `i++++` 是合法且常见写法

填空题

13. 逻辑与、或、非运算符分别是 __、__、__。
14. 动态分配单个对象用 __，释放用 __。
15. 动态分配数组用 __，释放用 __。
16. C++11 空指针字面量关键字是 __。
17. `a += 2;` 等价于 __。
18. 条件运算符的基本形式是：条件 ? 表达式1 : __。

读程序写结果

19. 写出程序输出：

```
#include <iostream>
int main() {
    int a = 5, b = 2;
    std::cout << a / b << " " << a % b << "\n";
    return 0;
}
```

20. 写出程序输出：

```
#include <iostream>
int main() {
    int x = 3;
    std::cout << ++x << " ";
    std::cout << x++ << " ";
    std::cout << x << "\n";
    return 0;
}
```

21. 写出程序输出：

```
#include <iostream>
int main() {
    bool r = (3 > 5) && (1 / 0 == 0); // 注意：右侧因短路不会求值
    std::cout << std::boolalpha << r << "\n";
    return 0;
}
```

22. 写出程序输出：

```
#include <iostream>
int main() {
    int* p = new int(7);
    std::cout << *p << "\n";
    delete p;
    return 0;
}
```

简答或编程

23. 说明何时使用 `delete` 与 `delete[]`，混用可能有什么后果。
24. 简述 `nullptr` 相对 `NULL` / `0` 的优点。
25. 写出用 `new` / `delete[]` 分配长度为 5 的 `int` 数组、全部赋值为下标并输出后释放的代码片段。

二、参考答案与解析

1. 答案：B

解析：整数除法向零截断，7/2 得 3。

2. 答案：B

解析：7 除以 2 余 1。

3. 答案：B

解析：C++ 关系运算结果为 `bool`（C 中为 `int`）。

4. 答案：B

解析：`||` 左真则短路；`&&` 左假则短路。

5. 答案：B

解析：对 `ostream` 重载后表示输出插入。

6. 答案：B

解析：`nullptr` 是类型安全的空指针字面量。

7. 答案：B

解析：`new` 单个对象配对 `delete`。

8. 答案：B

解析：`new[]` 必须配对 `delete[]`。

9. 答案：B

解析：对 `istream` 表示输入提取。

10. 答案：B

解析：赋值运算符右结合，先 `b=3` 再 `a=b`。

11. 答案：B

解析：用于命名空间、类作用域等，如 `std::cout`。

12. 答案：D

解析：不能对后置结果再 `++`，`i++++` 非法。

13. 答案：&&; ||; !

解析：逻辑运算符三件套。

14. 答案：new; delete

解析：成对使用，避免泄漏。

15. 答案：new[]; delete[]

解析：数组形式必须配对。

16. 答案：nullptr

解析：C++11 引入。

17. 答案：`a = a + 2;`

解析：复合赋值等价展开。

18. 答案：表达式2

解析：条件为真取前者，否则取后者。

19. 答案：

2 1

解析：整数除得 2，余数为 1。

20. 答案：

4 4 5

解析：先前置到 4 并输出；再后置输出 4 后变为 5；最后输出 5。

21. 答案：

false

解析：`(3>5)` 为假，`&&` 短路，不计算右侧，整体为 `false`。

22. 答案：

7

解析：`new int(7)` 值初始化为 7，解引用输出后 `delete`。

23. 答案要点：

`new` 配 `delete`；`new[]` 配 `delete[]`。混用导致未定义行为（可能只释放部分、析构不正确等）。

24. 答案要点：

`nullptr` 类型为 `std::nullptr_t`，不会与整型重载混淆；比宏 `NULL` 更类型安全。

25. 参考代码：

```
int* a = new int[5];
for (int i = 0; i < 5; ++i) a[i] = i;
for (int i = 0; i < 5; ++i) std::cout << a[i] << " ";
delete[] a;
```

三、本章知识点总结

要点	速记
算术	整除截断； <code>%</code> 取余
关系逻辑	结果为 <code>bool</code> ； <code>&&</code> / <code> </code> 短路
流运算	<code><<</code> 输出 / <code>>></code> 输入
作用域	<code>::</code>
动态内存	<code>new</code> / <code>delete</code> ， <code>new[]</code> / <code>delete[]</code>
空指针	推荐 <code>nullptr</code>

第 4 章 · 流程控制

本章知识点速览

- `if` / `else if` / `else` ; 条件可为 `bool`
- `switch` : 整型或枚举, 注意 `break` 防止贯穿
- 传统 `for` / `while` / `do-while` ; `break` 与 `continue`
- 范围 `for` (C++11) : `for (auto x : c)` / `auto&` / `const auto&`
- 条件运算符 `?:`

一、题目

选择题

1. 下列关于 `if` 的说法, 正确的是:
 - A. 条件必须是 `int`, 不能是 `bool`
 - B. `else` 总是匹配最近的尚未匹配的 `if`
 - C. `if` 后面不能跟复合语句
 - D. 条件中不能使用关系运算符
2. `switch` 的控制表达式通常不能是:
 - A. `int`
 - B. `char`
 - C. 枚举
 - D. `std::string` (标准 C++ 不支持直接 `switch` 字符串)
3. 范围 `for` `for (auto x : vec)` 中, 修改 `x` :
 - A. 一定修改容器元素
 - B. 一般只改拷贝, 不影响原元素
 - C. 语法错误
 - D. 等价于 `auto&`
4. 若要在范围 `for` 中修改容器元素, 应写成:
 - A. `for (auto x : vec)`
 - B. `for (auto& x : vec)`
 - C. `for (const auto& x : vec)`
 - D. `for (auto* x : vec)` (对 `vector<int>`)
5. `do-while` 相对 `while` 的特点是:
 - A. 可能一次也不执行循环体
 - B. 至少执行一次循环体
 - C. 不能使用 `break`
 - D. 条件写在开头
6. `continue` 的作用是:
 - A. 结束整个程序
 - B. 跳出当前循环
 - C. 结束本次迭代, 进入下一次循环判断/步进
 - D. 跳到 `switch` 的 `default`
7. 下列循环一定会无限循环的是 (忽略溢出) :
 - A. `for (int i = 0; i < 10; ++i) {}`
 - B. `while (false) {}`

- C. `for (::) {}`
D. `do { break; } while (true);`
8. 关于 `break`，正确的是：
A. 可跳出多重循环的所有层
B. 只跳出最近的一层循环或 `switch`
C. 只能用于 `for`
D. 与 `return` 完全相同
9. `const auto&` 用于范围 `for` 的主要目的是：
A. 强制修改元素
B. 只读访问并避免拷贝
C. 把元素变成指针
D. 改变容器大小
10. 表达式 `(a > b) ? a : b` 的含义是：
A. 永远返回 `a`
B. `a > b` 时取 `a`，否则取 `b`
C. 交换 `a` 与 `b`
D. 比较后返回 `bool`
11. `switch` 中缺少 `break` 通常会导致：
A. 编译错误
B. case 贯穿 (fall-through)
C. 自动返回 `main`
D. 清空变量
12. 传统 `for` `for (int i = 0; i < n; ++i)` 中，`i` 的作用域通常是：
A. 整个文件
B. 仅该 `for` 语句 (含循环体)
C. 仅初始化部分
D. 全局

填空题

13. 范围 `for` 的基本语法是：`for (声明 : ____)`。
14. `while` 先判断条件；`do-while` 先执行 __ 再判断。
15. 跳出循环用 __；跳过本次剩余语句用 __。
16. 只读遍历较大元素时，推荐 `for (____ auto& x : container)`。
17. `switch` 中匹配不到任何 `case` 时，会进入 __ 标签 (若有)。
18. C++11 起可用 __ `for` 遍历数组、`string`、`vector` 等。

读程序写结果

19. 写出程序输出：

```
#include <iostream>
int main() {
    int x = 75;
    if (x >= 90) std::cout << "A";
    else if (x >= 60) std::cout << "B";
    else std::cout << "F";
    std::cout << "\n";
    return 0;
}
```

20. 写出程序输出:

```
#include <iostream>
int main() {
    for (int i = 1; i <= 5; ++i) {
        if (i == 3) continue;
        std::cout << i;
    }
    std::cout << "\n";
    return 0;
}
```

21. 写出程序输出:

```
#include <iostream>
#include <vector>
int main() {
    std::vector<int> v = {1, 2, 3};
    for (auto& x : v) x *= 2;
    for (auto x : v) std::cout << x << " ";
    std::cout << "\n";
    return 0;
}
```

22. 写出程序输出:

```
#include <iostream>
int main() {
    int op = 1, a = 4, b = 2, r = 0;
    switch (op) {
        case 1: r = a + b; break;
        case 2: r = a - b; break;
        default: r = -1;
    }
    std::cout << r << "\n";
    return 0;
}
```

简答或编程

23. 说明范围 for 三种写法 `auto` / `auto&` / `const auto&` 的选用场景。
24. 用范围 for 计算 `std::vector<int> v = {1,2,3,4}` 的元素之和并输出。
25. 简述 `switch` 中 `break` 的作用及忘记书写的典型后果。

二、参考答案与解析

1. 答案: B

解析: `else` 就近匹配; 条件可为 `bool` 等可转上下文布尔的类型。

2. 答案: D

解析: 标准 `switch` 需要整型或枚举等; `std::string` 不能直接作为控制表达式。

3. 答案: B

解析: 按值是拷贝, 改 `x` 不影响容器。

4. 答案: B

解析: 引用才能绑定到元素并修改。

5. 答案: B

解析: `do-while` 后测条件, 至少执行一次。

6. 答案: C

解析: `continue` 结束本轮, 进入下一轮。

7. 答案: C

解析: `for (;;)` 条件恒真, 形成无限循环。

8. 答案: B

解析: 只影响最内层循环或 `switch`。

9. 答案: B

解析: 避免拷贝且禁止修改。

10. 答案: B

解析: 三元运算符按条件选值。

11. 答案: B

解析: 无 `break` 会继续执行后续 case。

12. 答案: B

解析: C++ 中 `for` 初始化声明的变量作用域限于该循环。

13. 答案: 表达式 (容器/数组等)

解析: 冒号右侧为被遍历的范围。

14. 答案: 循环体

解析: 先执行再判断。

15. 答案: `break; continue`

解析: 控制流基本工具。

16. 答案: `const`

解析: `const auto&` 只读免拷贝。

17. 答案: `default`

解析: 缺省分支。

18. 答案：范围 (range-based)

解析：C++11 范围 for。

19. 答案：

B

解析：75≥60 且 <90，输出 B。

20. 答案：

1245

解析：`i==3` 时 `continue`，跳过输出 3。

21. 答案：

2 4 6

解析：引用翻倍后再按值输出。

22. 答案：

6

解析：`op==1` 执行加法 4+2。

23. 答案要点：

`auto`：小对象拷贝或只需只读拷贝；`auto&`：需修改元素；`const auto&`：只读且避免大对象拷贝。

24. 参考代码：

```
#include <iostream>
#include <vector>
int main() {
    std::vector<int> v = {1, 2, 3, 4};
    int sum = 0;
    for (auto x : v) sum += x;
    std::cout << sum << "\n";
    return 0;
}
```

25. 答案要点：

`break` 结束 `switch`；缺少时会贯穿到后续 case，可能执行多余语句并得到错误结果。

三、本章知识点总结

要点	速记
分支	<code>if / else ;</code> <code>switch + break</code>
循环	<code>for / while / do-while</code>
范围 for	<code>auto / auto& / const auto&</code>
跳转	<code>break</code> 跳出; <code>continue</code> 下轮
条件运算	<code>cond ? a : b</code>

第 5 章 · 函数与重载

本章知识点速览

- 声明与定义分离；调用前需可见声明或定义
- 传参：值传递、引用传递、常引用、指针传递
- 函数重载：同名不同参数列表；仅返回值不同不能重载
- 默认参数：从右向左连续给出，通常写在声明处
- `inline` 建议内联；`constexpr` 函数可在编译期求值

一、题目

选择题

1. 下列关于函数声明与定义，正确的是：
A. 声明必须包含函数体
B. 定义提供函数体；声明告知签名
C. 二者完全相同
D. 只能定义不能声明
2. 值传递的特点是：
A. 函数内修改形参一定影响实参
B. 形参是实参的拷贝，修改一般不影响实参
C. 不能传递 `int`
D. 必须配合 `const`
3. 下列哪组函数构成合法重载？
A. `int f();` 与 `double f();`
B. `int f(int);` 与 `int f(double);`
C. `int f(int);` 与 `int f(int);` (完全相同)
D. 仅命名空间不同的两个同签名函数
4. 关于默认参数，错误的是：
A. 应从右侧开始连续指定
B. 通常在声明中给出默认值

- C. 可以只给中间某个参数默认值而左右都无默认
D. 调用时可省略带默认值的尾部实参
5. `void f(const std::string& s)` 的优点主要是:
A. 一定比指针慢
B. 避免拷贝且防止函数修改实参
C. 可以传入 `nullptr`
D. 不能接受临时 `string`
6. 仅返回类型不同能否构成重载?
A. 能
B. 不能
C. 仅当返回 `void` 时能
D. 仅模板可以
7. 关于 `inline`, 正确的是:
A. 强制编译器必须内联
B. 是给编译器的内联建议; 类内定义的成员函数通常隐式 `inline`
C. 只能用于 `main`
D. 与 `constexpr` 含义相同
8. 指针传递 `void f(int* p)` 相对引用传递, 特点是:
A. 不能修改所指对象
B. 可以传递“无对象”的情况 (`nullptr`)
C. 调用时写法必须完全相同
D. 不能与重载共存
9. 下列调用与声明 `void g(int a, int b = 2, int c = 3);` 匹配的是:
A. `g();`
B. `g(1);`
C. `g(, 2, 3);`
D. `g(1,,3);`
10. 函数重载解析主要依据:
A. 函数名所在源文件名
B. 实参类型与个数等与形参的匹配
C. 返回值类型唯一决定
D. 调用处的注释
11. `constexpr int sq(int n) { return n * n; }` 的特点是:
A. 只能在运行期调用
B. 在常量表达式语境下可于编译期求值
C. 不能返回 `int`
D. 等价于宏, 无类型检查
12. 下列关于引用传参 `void swap(int& a, int& b)`, 正确的是:
A. 调用写作 `swap(&x, &y);`
B. 调用写作 `swap(x, y);`, 函数内可交换实参
C. 无法交换两个变量
D. 必须返回交换后的值

填空题

13. 函数 __ 给出函数体; 函数 __ 通常只有签名并以分号结束。
14. 同名函数参数类型或个数不同称为函数 __。
15. 默认参数必须从 __ 侧开始连续指定。
16. 大对象只读传参推荐写法: `const T&`, 其中 `&` 表示 __ 传递。

17. 不能仅靠不同的 `__` 类型来重载函数。

18. 声明 `int add(int, int);` 中可省略 `__` 名, 但类型不能省。

读程序写结果

19. 写出程序输出:

```
#include <iostream>
int add(int a, int b) { return a + b; }
double add(double a, double b) { return a + b; }
int main() {
    std::cout << add(1, 2) << " ";
    std::cout << add(1.5, 2.5) << "\n";
    return 0;
}
```

20. 写出程序输出:

```
#include <iostream>
void f(int x) { x = 100; }
void g(int& x) { x = 100; }
int main() {
    int a = 1, b = 1;
    f(a);
    g(b);
    std::cout << a << " " << b << "\n";
    return 0;
}
```

21. 写出程序输出:

```
#include <iostream>
void greet(const char* name, const char* prefix = "Hi") {
    std::cout << prefix << " " << name;
}
int main() {
    greet("Ann");
    std::cout << "; ";
    greet("Bob", "Hello");
    std::cout << "\n";
    return 0;
}
```

22. 写出程序输出:

```
#include <iostream>
int scale(int x, int k = 2) { return x * k; }
int main() {
    std::cout << scale(5) << " " << scale(5, 3) << "\n";
    return 0;
}
```


简答或编程

23. 说明为什么“仅返回类型不同”不能构成重载。
24. 编写重载函数 `max`：分别接受两个 `int` 与两个 `double`，返回较大者；在 `main` 中各调用一次并输出。
25. 简述值传递、引用传递、常引用传递的选用原则。

二、参考答案与解析

1. 答案：B

解析：声明引入名字与签名；定义提供实现。

2. 答案：B

解析：值传递拷贝形参，一般不影响实参。

3. 答案：B

解析：参数类型不同可重载；仅返回类型不同不行。

4. 答案：C

解析：默认参数必须从右连续，不能“挖空”中间。

5. 答案：B

解析：常引用免拷贝且只读；也可绑定临时 `string`。

6. 答案：B

解析：重载不看返回类型，否则调用处无法区分。

7. 答案：B

解析：`inline` 是建议；类内定义成员常隐式 `inline`。

8. 答案：B

解析：指针可为空；引用通常应绑定有效对象。

9. 答案：B

解析：可省略尾部默认实参；不能空置中间参数。

10. 答案：B

解析：根据实参与候选形参匹配进行重载决议。

11. 答案：B

解析：满足条件时可作编译期常量表达式。

12. 答案：B

解析：引用实参直接写变量名，无需取地址。

13. 答案：定义；声明

解析：定义有函数体，声明以 `;` 结束。

14. 答案：重载

解析：C++ 支持，C 不支持。

15. 答案：右

解析：从右向左连续默认。

16. 答案：引用

解析：`&` 表示引用传递。

17. 答案：返回

解析：返回类型不参与重载特征。

18. 答案：参数（形参）

解析：声明中参数名可选。

19. 答案：

3 4

解析：分别匹配 `int` 与 `double` 重载。

20. 答案：

1 100

解析：`f` 值传递不影响 `a`；`g` 引用修改 `b`。

21. 答案：

Hi Ann; Hello Bob

解析：默认前缀 `Hi`；第二次显式传入 `Hello`。

22. 答案：

10 15

解析：默认 `k=2` 得 10；显式 `k=3` 得 15。

23. 答案要点：

调用处常常不依赖返回值参与决议（甚至忽略返回值），若仅返回类型不同，编译器无法根据调用确定调用哪个函数。

24. 参考代码：

```
#include <iostream>
int max(int a, int b) { return (a > b) ? a : b; }
double max(double a, double b) { return (a > b) ? a : b; }
int main() {
    std::cout << max(3, 5) << " " << max(2.5, 1.2) << "\n";
    return 0;
}
```

25. 答案要点:

小对象或无需改实参: 值传递; 需修改实参: 引用 (或指针) ; 大对象只读: `const T&`。

三、本章知识点总结

要点	速记
声明/定义	声明签名; 定义含函数体
传参	值 / 引用 / <code>const T&</code> / 指针
重载	参数列表不同; 不看返回类型
默认参数	从右连续; 多写在声明
inline/constexpr	内联建议; 编译期求值

第 6 章 · 引用与指针

本章知识点速览

- 引用: 别名, 必须初始化; 常引用只读
- 指针: 存地址; `*` 解引用; `&` 取址; 可为 `nullptr`
- `new` / `delete` 与 `new[]` / `delete[]` 成对使用
- 智能指针概要: `unique_ptr` / `shared_ptr` / `weak_ptr` (`<memory>`)
- 引用适合别名与函数参数; 指针适合可空、可重新指向、动态数组等

一、题目

选择题

1. 关于引用, 错误的是:
A. 必须初始化
B. 是已存在对象的别名
C. 可以像指针一样做指针算术
D. 常引用可用 `const T&`
2. `int* p = nullptr;` 表示:
A. `p` 指向地址 0 的合法对象
B. `p` 为空指针, 不指向有效对象

- C. `p` 自动变成引用
D. 语法错误
3. `int x = 3; int* p = &x;` 则 `*p` 的值是:
A. `x` 的地址
B. 3
C. `p` 的地址
D. 不确定
4. 下列释放正确的是:
A. `int* p = new int[5]; delete p;`
B. `int* p = new int[5]; delete[] p;`
C. `int* p = new int; delete[] p;`
D. `int* p = new int; free(p);` (推荐写法)
5. 关于 `unique_ptr`, 正确的是:
A. 多个 `unique_ptr` 默认可共享同一对象
B. 独占所有权, 通常不可拷贝、可移动
C. 不需要头文件
D. 与 `shared_ptr` 完全相同
6. `shared_ptr` 的主要特点是:
A. 弱引用不计数
B. 共享所有权, 使用引用计数
C. 不能指向堆对象
D. 比裸指针更不安全
7. 下列哪项通常用于打破 `shared_ptr` 循环引用?
A. `unique_ptr`
B. `weak_ptr`
C. 原始数组
D. `auto`
8. 引用与指针相比, 正确的是:
A. 引用可以稍后改为绑定别的对象 (一般语义)
B. 指针可以重新指向其他对象
C. 引用可以为空, 指针不能
D. 二者语法完全一样
9. `const int* p` 与 `int* const p` 的区别, 正确的是:
A. 完全相同
B. 前者指向的对象只读 (底层 `const`), 后者指针本身不可变 (顶层 `const`)
C. 前者指针不可变, 后者对象只读
D. 都表示指针与对象都可变
10. `int& r = x;` 之后执行 `r = 10;` 的效果是:
A. 让 `r` 改绑到字面量 10
B. 把 `x` 的值改为 10
C. 编译错误
D. 只改 `r` 的“指针值”
11. 推荐创建 `unique_ptr<int>` 的方式是:
A. `std::make_unique<int>(42);` (C++14)
B. 必须手写 `new` 且不能封装
C. `malloc(sizeof(int))`
D. `std::weak_ptr<int>(42)`

12. 下列关于解引用空指针的说法，正确的是：

- A. 结果固定为 0
- B. 属于未定义行为，应避免
- C. 编译器一定报错并拒绝生成程序
- D. 会自动分配对象

填空题

13. 声明整型引用： `int _____ r = x;`。
14. 取变量 `x` 的地址： `int* p = _____ x;`。
15. 空指针字面量： `p = _____;`。
16. `new` 单个对象用 `_____` 释放； `new[]` 用 `_____` 释放。
17. `unique_ptr` / `shared_ptr` 头文件是 `_____`。
18. 通过指针修改 `x`：已知 `int* p = &x;`，应写 `_____ = 5;`。

读程序写结果

19. 写出程序输出：

```
#include <iostream>
int main() {
    int x = 10;
    int& r = x;
    int* p = &x;
    r = 20;
    *p = *p + 1;
    std::cout << x << " " << r << " " << *p << "\n";
    return 0;
}
```

20. 写出程序输出：

```
#include <iostream>
void set(int* p) {
    if (p) *p = 42;
}
int main() {
    int a = 0;
    set(&a);
    set(nullptr);
    std::cout << a << "\n";
    return 0;
}
```

21. 写出程序输出：

```
#include <iostream>
int main() {
    int* arr = new int[3]{1, 2, 3};
    std::cout << arr[0] + arr[1] + arr[2] << "\n";
    delete[] arr;
    return 0;
}
```

22. 写出程序输出:

```
#include <iostream>
#include <memory>
int main() {
    auto p = std::make_shared<int>(9);
    auto q = p;
    std::cout << *q << " " << p.use_count() << "\n";
    return 0;
}
```

简答或编程

23. 比较引用参数与指针参数：何时更倾向引用？何时必须用指针？
24. 说明 `new` / `delete` 与智能指针在资源管理上的差异（至少两点）。
25. 写代码：用 `new` 分配一个 `int` 并初始化为 5，输出后正确释放。

二、参考答案与解析

1. 答案：C

解析：引用不是指针，不能做指针算术。

2. 答案：B

解析：`nullptr` 表示空，解引用非法。

3. 答案：B

解析：`*p` 解引用得到 `x` 的值 3。

4. 答案：B

解析：数组形式必须 `delete[]`。

5. 答案：B

解析：`unique_ptr` 独占，移动语义转移所有权。

6. 答案：B

解析：共享所有权靠引用计数管理寿命。

7. 答案：B

解析：`weak_ptr` 不增加引用计数，可破坏。

8. 答案: B

解析: 指针可重新指向; 引用一般不可改绑。

9. 答案: B

解析: `const` 位置决定修饰指针还是对象。

10. 答案: B

解析: 对引用赋值即对原对象赋值。

11. 答案: A

解析: `make_unique` 更安全便捷 (C++14)。

12. 答案: B

解析: 解引用空指针是未定义行为。

13. 答案: &

解析: `int& r = x;`。

14. 答案: &

解析: 取址运算符。

15. 答案: `nullptr`

解析: 推荐空指针字面量。

16. 答案: `delete; delete[]`

解析: 必须配对。

17. 答案: `<memory>`

解析: 智能指针标准头。

18. 答案: `*p`

解析: 解引用后赋值。

19. 答案:

21 21 21

解析: 引用改为 20, 再通过指针加 1, 三者皆为 21。

20. 答案:

42

解析: 第一次设置 ``a``; 第二次空指针被判断跳过。

21. 答案:

6

解析：1+2+3=6，随后 `delete[]`。

22. 答案：

9 2

解析：共享同一对象，引用计数为 2。

23. 答案要点：

参数非空且需别名语义时优先引用，调用更简洁；需要表示“可选/空”、要在函数内改变指针本身指向、或与 C API 交互时用指针。

24. 答案要点：

裸 `new` / `delete` 易忘记释放或异常路径泄漏；智能指针利用 RAII 自动释放，所有权语义更清晰（独占/共享）。

25. 参考代码：

```
#include <iostream>
int main() {
    int* p = new int(5);
    std::cout << *p << "\n";
    delete p;
    return 0;
}
```

三、本章知识点总结

要点	速记
引用	别名，必须初始化
指针	地址 + 解引用；可 <code>nullptr</code>
堆内存	<code>new</code> / <code>delete</code> , <code>new[]</code> / <code>delete[]</code>
智能指针	<code>unique_ptr</code> 独占； <code>shared_ptr</code> 共享； <code>weak_ptr</code> 弱引用
选型	非空别名→引用；可空/可改指→指针

第 7 章 · 数组与 string

本章知识点速览

- C 风格数组: `int a[5]`; C++11 列表初始化 `int a[]{1,2,3}`
- `std::array<T,N>`: 固定长度, 有 `size()`
- `std::string`: `size` / `+` / `[]` / `at` / `substr` / `find`; 头文件 `<string>`
- `std::vector`: 动态数组, `push_back` / `pop_back` / `size` / `[]`; 头文件 `<vector>`
- 可用范围 `for` 遍历数组、`string`、`vector`

一、题目

选择题

- 关于 C 风格数组, 正确的是:
 - 可作为值直接用 `=` 整体赋值给另一数组 (数组变量之间)
 - 名字在多数表达式中会衰减为指向首元素的指针
 - 一定知道自己的 `size()` 成员
 - 下标越界编译器保证抛异常
- `std::array<int, 5>` 相对 C 数组的优点是:
 - 长度可变
 - 提供 `size()` 等接口且不会轻易衰减为指针
 - 不能用于范围 `for`
 - 不需要头文件
- `std::string s = "Hi"; s += "!";` 之后 `s` 是:
 - "Hi"
 - "Hi!"
 - !"
 - 编译错误
- `s.size()` 与 `s.length()` 对 `std::string`:
 - 含义通常相同, 都返回字符个数
 - `length` 含结尾 `\0` 多 1
 - 只能用其中一个, 另一个非法
 - 返回字节容量 `capacity`
- `s.at(i)` 相对 `s[i]` 的常见区别是:
 - `at` 更慢且总是越界不检查
 - `at` 越界通常抛异常, `[]` 不检查
 - `[]` 不能用于非 `const string`
 - 二者返回类型不同到不能互换使用
- 关于 `std::vector`, 正确的是:
 - 长度编译期固定, 不能 `push_back`
 - 可动态增长, 常用 `push_back` 追加
 - 只能存 `char`
 - 没有 `size()`
- `v.pop_back()` 的作用是:
 - 删除首元素
 - 删除末元素
 - 清空全部元素
 - 返回并删除随机元素
- 下列初始化 `vector` 的写法, 正确的是:
 - `std::vector<int> v = {1, 2, 3};`
 - `std::vector<int> v = (1, 2, 3);`

- C. `std::vector v<> = 1,2,3;`
D. `vector = {1,2,3};` (无类型)
9. `std::string` 的 `find` 找不到子串时, 通常返回:
A. 0
B. -1 转成的 `size_t` 最大值, 即 `std::string::npos`
C. 空字符串
D. 抛出 `int` 异常 (必定)
10. 范围 `for` 遍历 `string` 时, `for (char c : s):`
A. 非法
B. 按字符依次访问拷贝
C. 自动按单词分割
D. 只能得到首字符
11. `std::array` 需要包含的头文件是:
A. `<array>`
B. `<vector>`
C. `<list>`
D. `<cstdio>`
12. 下列关于 `substr` 的说法, 正确的是:
A. `s.substr(pos, len)` 返回从 `pos` 起长度为 `len` 的子串
B. `substr` 会修改原字符串为子串
C. 只能截取单个字符
D. 不存在该成员函数

填空题

13. `std::string` 头文件是 __; `std::vector` 头文件是 __。
14. 向 `vector` 末尾添加元素用 __; 删除末尾用 __。
15. 获取 `string` / `vector` 元素个数常用成员函数 __。
16. C++11 固定长度数组包装器是 `std::_____<T, N>`。
17. `string` 拼接可用运算符 __ 或成员 `+=`。
18. 判断 `find` 失败可与常量 __ 比较。

读程序写结果

19. 写出程序输出:

```
#include <iostream>
#include <string>
int main() {
    std::string s = "C++";
    s += "17";
    std::cout << s << " " << s.size() << "\n";
    return 0;
}
```

20. 写出程序输出:

```
#include <iostream>
#include <vector>
int main() {
    std::vector<int> v = {1, 2};
    v.push_back(3);
    v.pop_back();
    std::cout << v.size() << " " << v[0] << v[1] << "\n";
    return 0;
}
```

21. 写出程序输出:

```
#include <iostream>
#include <string>
int main() {
    std::string s = "abcdef";
    std::cout << s.substr(2, 3) << "\n";
    return 0;
}
```

22. 写出程序输出:

```
#include <iostream>
#include <array>
int main() {
    std::array<int, 3> a = {2, 4, 6};
    int sum = 0;
    for (auto x : a) sum += x;
    std::cout << sum << " " << a.size() << "\n";
    return 0;
}
```

简答或编程

23. 对比 C 风格字符串 (`char[]`) 与 `std::string` 在长度、拼接、安全性上的差异。
24. 用 `vector` 读入思路: 创建空 `vector<int>`, `push_back` 三个数 10、20、30, 用范围 `for` 输出。
25. 简述何时用 `std::array`、何时用 `std::vector`。

二、参考答案与解析

1. 答案: B

解析: 数组名常衰减为指针; 不能整体赋值; 无 `size()` 成员; 越界是未定义行为。

2. 答案: B

解析: `std::array` 是对象, 保留长度信息, 接口更安全清晰。

3. 答案: B

解析: `+=` 追加字符串。

4. 答案: A

解析: `size` 与 `length` 同义, 返回字符数 (不含 C 风格那种需自己数 `\0` 的心智负担由类管理)。

5. 答案: B

解析: `at` 做边界检查, 失败抛 `std::out_of_range`。

6. 答案: B

解析: `vector` 是动态数组容器。

7. 答案: B

解析: 弹出最后一个元素。

8. 答案: A

解析: 列表初始化是常见写法。

9. 答案: B

解析: 返回 `npos` 表示未找到。

10. 答案: B

解析: 逐个字符遍历。

11. 答案: A

解析: `<array>` 提供 `std::array`。

12. 答案: A

解析: `substr` 返回新串, 不修改原串 (除非再赋值回去)。

13. 答案: `<string>; <vector>`

解析: 分别引入对应标准库组件。

14. 答案: `push_back; pop_back`

解析: 尾部增删。

15. 答案: `size` (或 `string` 的 `length`)

解析: 元素/字符个数。

16. 答案: `array`

解析: `std::array<T,N>`。

17. 答案: `+`

解析: `s1 + s2` 或 `s1 += s2`。

18. 答案: `std::string::npos`

解析: 未找到的标志值。

19. 答案:

解析：拼接后长度为 5。

20. 答案：

2 12

解析：push 后 size=3, 再 pop 回到 2, 元素为 1、2。

21. 答案：

cde

解析：从下标 2 起取 3 个字符。

22. 答案：

12 3

解析：2+4+6=12, `size()` 为 3。

23. 答案要点：

C 字符串靠 `\0` 结尾，长度需 `strlen`，拼接易溢出；`std::string` 管理长度与内存，支持 `+`/`+=`，使用更安全方便。

24. 参考代码：

```
#include <iostream>
#include <vector>
int main() {
    std::vector<int> v;
    v.push_back(10);
    v.push_back(20);
    v.push_back(30);
    for (auto x : v) std::cout << x << " ";
    std::cout << "\n";
    return 0;
}
```

25. 答案要点：

长度编译期固定、需栈上语义/接口安全时用 `std::array`；运行期需要动态增减用 `std::vector`。

三、本章知识点总结

要点	速记
C 数组	固定长度，易衰减为指针
std::array	<array>，size()，固定 N
std::string	拼接/ substr / find / at
std::vector	动态数组，push_back / pop_back
遍历	范围 for 适用于三者

第 8 章 · 类与对象

本章知识点速览

- class 将数据与操作封装；成员默认 private（相对 struct 默认 public）
- 构造函数同名无返回类型；可用初始化列表 : member(x)；析构函数 ~ClassName()
- const 成员函数不修改对象；static 成员属类共享；this 指向当前对象
- 访问控制：public / protected / private 决定类外与派生类可见性

一、题目

选择题

- 关于 C++ 中 class 与 struct，正确的是：
 - 二者完全相同，无任何差别
 - class 成员默认 public，struct 默认 private
 - class 成员默认 private，struct 默认 public
 - struct 不能有成员函数
- 下列关于构造函数的说法，错误的是：
 - 构造函数名与类名相同
 - 构造函数可以有返回类型 void
 - 构造函数在创建对象时自动调用
 - 一个类可以有多个重载构造函数
- 初始化列表 : member(x) 的主要作用是：
 - 在函数体内赋值，与 member = x 完全等价且无差别
 - 在构造对象成员时直接初始化，适合 const 成员与引用成员
 - 仅用于静态成员
 - 只能初始化公有成员
- 析构函数的正确写法是：
 - void ~Foo()
 - ~Foo()
 - Foo~()
 - delete Foo()
- 关于 const 成员函数，正确的是：
 - 可以随意修改所有非静态数据成员
 - 承诺不修改对象的逻辑状态（通常不改非 mutable 成员）

- C. 只能在类外定义，不能在类内定义
 - D. 与普通成员函数调用方式完全不同
6. 静态数据成员的特点是：
- A. 每个对象各有一份独立拷贝
 - B. 属于类，所有对象共享一份
 - C. 必须是 `private`
 - D. 不能在类外定义
7. 在非静态成员函数中，`this` 的含义是：
- A. 指向当前类类型信息的指针
 - B. 指向当前对象的指针
 - C. 指向基类对象的引用
 - D. 指向静态成员的指针
8. 下列访问控制描述，正确的是：
- A. `private` 成员可被类外直接访问
 - B. `protected` 成员可被类外任意代码访问
 - C. `public` 成员可被类外访问
 - D. 派生类不能访问基类任何成员
9. 若类 `A` 未显式定义任何构造函数，编译器通常会：
- A. 报错，禁止创建对象
 - B. 生成默认构造函数（在可生成的前提下）
 - C. 自动生成带三个 `int` 参数的构造函数
 - D. 禁止使用 `new`
10. 下列哪项适合放在类的 `private` 区？
- A. 对外提供的接口函数
 - B. 仅供类内部使用的数据成员与辅助函数
 - C. 必须被全局函数直接修改的变量
 - D. `main` 函数

填空题

- 11. 类定义中，公有访问区用关键字 __，私有访问区用 __。
- 12. 析构函数名是在类名前加字符 __。
- 13. 不修改对象状态的成员函数通常在参数列表后写 __。
- 14. 静态成员函数中 __（能/不能）直接访问非静态数据成员。
- 15. 构造函数若要在创建对象时直接初始化成员，常用 __ 列表。

读程序写结果

- 16. 写出程序输出：

```

#include <iostream>
class Point {
public:
    Point(int x, int y) : x_(x), y_(y) {}
    void print() const { std::cout << x_ << ", " << y_; }
private:
    int x_, y_;
};
int main() {
    Point p(3, 4);
    p.print();
    return 0;
}

```

17. 写出程序输出:

```

#include <iostream>
class Counter {
public:
    Counter() { ++count; }
    static int get() { return count; }
private:
    static int count;
};
int Counter::count = 0;
int main() {
    Counter a, b, c;
    std::cout << Counter::get();
    return 0;
}

```

18. 写出程序输出:

```

#include <iostream>
class Foo {
public:
    Foo() { std::cout << "C"; }
    ~Foo() { std::cout << "D"; }
};
int main() {
    { Foo f; }
    std::cout << "E";
    return 0;
}

```

19. 写出程序输出:

```

#include <iostream>
class Box {
public:
    Box(int n) : n_(n) {}

```



```

    int get() const { return n_; }
    void set(int n) { n_ = n; }
private:
    int n_;
};
int main() {
    Box b(10);
    b.set(20);
    std::cout << b.get();
    return 0;
}

```

20. 写出程序输出：

```

#include <iostream>
class A {
public:
    A(int v) : v_(v) {}
    int v() const { return v_; }
private:
    int v_;
};
int main() {
    A a(5);
    A* p = &a;
    std::cout << p->v();
    return 0;
}

```

简答或编程

- 简述封装的含义，并说明为何常把数据成员设为 `private`、接口设为 `public`。
- 说明构造函数与析构函数各自何时被调用，并各举一个典型用途。
- 编写一个类 `Rectangle`，含长宽私有成员、带参构造、`area()` 常成员函数；在 `main` 中创建对象并输出面积。
- 解释 `this` 指针的用途，并说明静态成员函数为何通常没有 `this`。

二、参考答案与解析

1. 答案：C

解析：默认访问权限是二者主要差别之一：`class` 默认私有，`struct` 默认公有。

2. 答案：B

解析：构造函数不能写返回类型（包括 `void`）。

3. 答案：B

解析：初始化列表在进入函数体前初始化成员；`const` 引用成员必须用它（或类内初值）。

4. 答案：B

解析：析构函数写作 `~类名()`，无返回类型、通常无参数。

5. 答案: B

解析: `const` 成员函数不应修改对象非 `mutable` 状态, 可被常对象调用。

6. 答案: B

解析: 静态数据成员属于类, 一份共享。

7. 答案: B

解析: `this` 是指向当前对象的指针。

8. 答案: C

解析: `public` 对外可见; `private` / `protected` 对类外 (非友元) 不可直接访问。

9. 答案: B

解析: 未声明构造函数时, 在可生成条件下会有隐式默认构造函数。

10. 答案: B

解析: 私有区用于隐藏实现细节。

11. 答案: **public; private**

解析: 访问说明符划分可见性。

12. 答案: ~

解析: 析构造函数名为 `~ClassName`。

13. 答案: **const**

解析: `void f() const;` 表示常成员函数。

14. 答案: 不能

解析: 静态成员函数无 `this`, 不能直接访问非静态成员。

15. 答案: **初始化 (成员初始化)**

解析: `: member(x)` 即成员初始化列表。

16. 答案:

3,4

解析: 构造时用初始化列表设置坐标并打印。

17. 答案:

3

解析: 创建三个对象, 构造函数使静态计数变为 3。

18. 答案:

CDE

解析：进入块构造输出 `C`，离开块析构输出 `D`，然后输出 `E`。

19. 答案：

20

解析：`set(20)` 后 `get()` 返回 20。

20. 答案：

5

解析：通过对象指针调用常成员函数。

21. 答案要点：

封装把数据与操作捆在一起并隐藏实现；`private` 防止外部随意改内部状态，`public` 提供稳定接口，便于维护与不变式保护。

22. 答案要点：

构造：对象创建时调用，用于初始化资源/成员；析构：对象销毁时调用，用于释放资源（关闭文件、`delete` 等）。

23. 参考代码：

```
#include <iostream>
class Rectangle {
public:
    Rectangle(double w, double h) : w_(w), h_(h) {}
    double area() const { return w_ * h_; }
private:
    double w_, h_;
};
int main() {
    Rectangle r(3.0, 4.0);
    std::cout << r.area() << "\n";
    return 0;
}
```

24. 答案要点：

`this` 指向当前对象，用于区分成员与形参、返回自身引用等；静态成员函数不绑定具体对象，因此没有 `this`。

三、本章知识点总结

要点	速记
默认访问	<code>class</code> 默认 <code>private</code>
构造/析构	同名/ <code>~名</code> ，无返回类型
初始化列表	<code>: mem(x)</code> ，适合 <code>const</code> /引用
<code>const</code> 成员	<code>f() const</code> 不改对象
<code>static</code>	类共享，无 <code>this</code>
访问控制	<code>public</code> / <code>protected</code> / <code>private</code>

第 9 章 · 继承与多态

本章知识点速览

- 继承语法: `class Derived : public Base { ... };`
- 虚函数 `virtual` + 派生类 `override`；基类析构建议虚析构
- 纯虚函数 `= 0` 使类成为抽象类，不能实例化
- 多态：基类指针/引用调用虚函数时按**实际对象类型**绑定

一、题目

选择题

1. 下列正确的公有继承写法是：
 - A. `class D extends B`
 - B. `class D : public B`
 - C. `class D inherit B`
 - D. `class D(B)`
2. 关于虚函数，正确的是：
 - A. 虚函数不能被派生类重写
 - B. 用基类指针调用虚函数时，通常按实际对象类型决定调用版本
 - C. 虚函数必须是静态的
 - D. 虚函数不能有参数
3. `override` 关键字的作用是：
 - A. 强制隐藏基类所有成员
 - B. 标明意图重写基类虚函数，签名不匹配时报错
 - C. 把函数变成内联
 - D. 禁止派生
4. 纯虚函数的写法是：
 - A. `virtual void f() pure;`
 - B. `virtual void f() = 0;`
 - C. `void f() virtual = 0;`
 - D. `abstract void f();`

5. 含有纯虚函数且未全部实现的类：
- A. 可以正常创建对象
 - B. 是抽象类，不能直接实例化
 - C. 自动变成 `final`
 - D. 只能有静态成员
6. 为何基类析构函数常声明为 `virtual` ？
- A. 提高构造速度
 - B. 通过基类指针 `delete` 派生对象时能正确调用派生析构
 - C. 禁止栈上创建对象
 - D. 让析构函数可以有参数
7. 下列关于多态的说法，错误的是：
- A. 多态依赖虚函数机制
 - B. 非虚函数通过基类指针调用时按指针静态类型绑定
 - C. 只有值传递对象才能产生多态
 - D. 基类引用调用虚函数也可体现多态
8. `protected` 继承下，基类的 `public` 成员在派生类中对外表现为：
- A. 仍为 `public`
 - B. 变为 `protected`
 - C. 变为 `private`
 - D. 不可访问
9. 派生类对象构造时，正确的调用顺序通常是：
- A. 先派生构造，再基类构造
 - B. 先基类构造，再派生构造
 - C. 只调用派生构造
 - D. 随机顺序
10. 下列哪项能阻止进一步继承？
- A. `virtual class`
 - B. `final`（用在类上）
 - C. `override`
 - D. `static`

填空题

11. 声明虚函数使用关键字 __。
12. 纯虚函数使类成为 __ 类。
13. 派生类重写虚函数时，C++11 可用 __ 明确标注。
14. 多态通常通过基类的 __ 或 __ 调用虚函数实现。
15. 通过基类指针释放派生对象时，基类析构应声明为 __。

读程序写结果

16. 写出程序输出：

```
#include <iostream>
class Base {
public:
    virtual void f() { std::cout << "B"; }
    virtual ~Base() = default;
};

class Derived : public Base {
```

```

public:
    void f() override { std::cout << "D"; }
};
int main() {
    Base* p = new Derived;
    p->f();
    delete p;
    return 0;
}

```

17. 写出程序输出:

```

#include <iostream>
class Base {
public:
    void f() { std::cout << "B"; }
};
class Derived : public Base {
public:
    void f() { std::cout << "D"; }
};
int main() {
    Derived d;
    Base* p = &d;
    p->f();
    return 0;
}

```

18. 写出程序输出:

```

#include <iostream>
class A {
public:
    A() { std::cout << "A"; }
    ~A() { std::cout << "a"; }
};
class B : public A {
public:
    B() { std::cout << "B"; }
    ~B() { std::cout << "b"; }
};
int main() {
    B x;
    return 0;
}

```

19. 写出程序输出:

```

#include <iostream>
class Animal {
public:
    virtual void speak() { std::cout << "??"; }
}

```

```

    virtual ~Animal() = default;
};
class Dog : public Animal {
public:
    void speak() override { std::cout << "woof"; }
};
void call(Animal& a) { a.speak(); }
int main() {
    Dog d;
    call(d);
    return 0;
}

```

20. 写出程序输出（说明能否编译；若能，写出输出）：

```

#include <iostream>
class Shape {
public:
    virtual double area() const = 0;
    virtual ~Shape() = default;
};
int main() {
    // Shape s;
    std::cout << "ok";
    return 0;
}

```

简答或编程

- 简述虚函数与普通成员函数在“通过基类指针调用”时的绑定差异。
- 什么是抽象类？给出一个含纯虚函数的接口类示例（可只写类定义）。
- 编写 `Base` / `Derived`，`Base` 有虚函数 `show()`，`Derived` 用 `override` 重写；用基类指针调用并输出。
- 说明若不把基类析构设为虚函数，用 `Base* p = new Derived; delete p;` 可能有什么问题。

二、参考答案与解析

1. 答案：B

解析：C++ 公有继承写作 `class D : public B`。

2. 答案：B

解析：虚函数实现运行时多态，按动态类型分派。

3. 答案：B

解析：`override` 帮助编译器检查是否真正重写了虚函数。

4. 答案：B

解析：纯虚函数语法为 `= 0`。

5. 答案：B

解析：抽象类不能直接实例化。

6. 答案: B

解析: 虚析构保证正确析构完整对象。

7. 答案: C

解析: 多态需要指针/引用 + 虚函数; 按值会切片, 反而失去多态。

8. 答案: B

解析: `protected` 继承会将基类 `public` 成员在派生类外可见性降为 `protected`。

9. 答案: B

解析: 先构造基类子对象, 再构造派生类。

10. 答案: B

解析: `final` 可用于类或虚函数, 阻止再继承/再重写。

11. 答案: `virtual`

解析: 虚函数关键字。

12. 答案: 抽象

解析: 含纯虚函数的类为抽象类。

13. 答案: `override`

解析: C++11 标注重写。

14. 答案: 指针; 引用

解析: 二者保留动态类型信息。

15. 答案: `virtual` (虚析构)

解析: 保证派生析构被调用。

16. 答案:

D

解析: 虚函数按实际类型 ``Derived`` 调用。

17. 答案:

B

解析: 非虚函数按静态类型 ``Base*`` 调用 ``Base::f``。

18. 答案:

ABba

解析：构造 $A \rightarrow B$ ，析构相反 $b \rightarrow a$ 。

19. 答案：

woof

解析：引用参数触发虚调用到 ``Dog::speak``。

20. 答案：

ok

解析：``Shape s;`` 被注释，抽象类未实例化，程序可运行输出 ``ok``。

21. 答案要点：

虚函数：运行时按对象真实类型调用；非虚函数：编译期按指针/引用的静态类型调用。

22. 答案要点：

含至少一个纯虚函数的类称抽象类，不能实例化。示例：

```
class Shape {
public:
    virtual double area() const = 0;
    virtual ~Shape() = default;
};
```

23. 参考代码：

```
#include <iostream>
class Base {
public:
    virtual void show() const { std::cout << "Base\n"; }
    virtual ~Base() = default;
};
class Derived : public Base {
public:
    void show() const override { std::cout << "Derived\n"; }
};
int main() {
    Base* p = new Derived;
    p->show();
    delete p;
    return 0;
}
```

24. 答案要点：

可能只调用基类析构，派生类资源未释放，造成资源泄漏或未定义行为。

三、本章知识点总结

要点	速记
继承	<code>class D : public B</code>
虚函数	<code>virtual</code> + 动态绑定
override	检查重写签名
纯虚/抽象	<code>= 0</code> ，不可实例化
多态条件	继承 + 虚函数 + 指针/引用
虚析构	<code>delete</code> 基类指针时必需

第 10 章 · 模板

本章知识点速览

- 函数模板: `template<typename T> T maximum(T a, T b);`
- 类模板: `template<typename T> class Box { ... };`
- `template<typename T>` 与 `template<class T>` 在类型参数上等价
- 模板是编译期多态/泛型，按实参生成具体实例

一、题目

选择题

- 下列正确的函数模板开头是：
A. `generic <T>`
B. `template<typename T>`
C. `template(T)`
D. `#define template T`
- 关于 `template<typename T>` 与 `template<class T>`，正确的是：
A. 完全不能互换
B. 在模板类型参数中二者等价
C. 只有 `class` 合法
D. `typename` 只能用于类模板
- 函数模板的主要目的是：
A. 运行时动态加载代码
B. 用同一套逻辑处理多种类型（编译期生成）
C. 替代所有虚函数
D. 只能用于 `int`
- 类模板实例化时，正确用法类似：
A. `Box b;`（无需类型实参，永远可以）

- B. `Box<int> b(1);`
C. `Box<> b;`
D. `new Box;`
5. 模板通常在何时完成类型检查与代码生成?
A. 链接之后
B. 运行时
C. 编译期 (实例化时)
D. 预处理之前
6. 下列关于函数模板实参推导, 正确的是:
A. 调用时绝不能省略类型实参
B. 常可由调用实参自动推导 `T`
C. 只能推导为 `int`
D. 推导失败也不会报错
7. 若写 `template<typename T> T add(T a, T b);`, 调用 `add(1, 2.5)` 通常会:
A. 一定成功, `T` 变成 `double`
B. 可能因 `T` 无法唯一推导而失败 (需显式指定或统一类型)
C. 运行时自动转换无编译检查
D. 变成宏展开
8. 类模板的成员函数若在类外定义, 通常需要:
A. 再次写 `template<...>` 并带上类模板限定
B. 不能在类外定义
C. 必须用宏
D. 只能定义静态成员
9. 模板与宏相比, 更优的一点通常是:
A. 模板没有类型检查
B. 模板有更好的类型安全与作用域规则
C. 宏能调试得更好且总是更安全
D. 模板只在运行时展开
10. 下列哪项不是模板的典型用途?
A. 泛型容器如 `vector<T>`
B. 泛型算法如 `max`
C. 替代 `#include`
D. 编写与类型无关的工具类

填空题

11. 函数模板以关键字 `__` 开头。
12. 类型参数既可写 `typename T`, 也可写 `__ T`。
13. `Box<int>` 中的 `int` 称为模板 `__`。
14. 模板实现的“一套代码多种类型”属于 `__` 期多态 (相对虚函数运行时多态)。
15. 显式指定函数模板实参的示例写法: `maximum_____ (3, 4)` (填 `<>` 中内容及相关符号)。

读程序写结果

16. 写出程序输出:

```
#include <iostream>
template<typename T>
T maximum(T a, T b) { return (a > b) ? a : b; }
int main() {
    std::cout << maximum(3, 7);
    return 0;
}
```

17. 写出程序输出:

```
#include <iostream>
template<typename T>
class Box {
    T content;
public:
    Box(T c) : content(c) {}
    T get() const { return content; }
};
int main() {
    Box<int> b(42);
    std::cout << b.get();
    return 0;
}
```

18. 写出程序输出:

```
#include <iostream>
template<typename T>
T square(T x) { return x * x; }
int main() {
    std::cout << square(4) << " " << square(1.5);
    return 0;
}
```

19. 写出程序输出:

```
#include <iostream>
#include <string>
template<typename T>
void print2(T a, T b) {
    std::cout << a << ", " << b;
}
int main() {
    print2<std::string>(std::string("A"), std::string("B"));
    return 0;
}
```

20. 写出程序输出:

```
#include <iostream>
template<typename T>
class Pair {
public:
    T first, second;
    Pair(T a, T b) : first(a), second(b) {}
};
int main() {
    Pair<char> p('x', 'y');
    std::cout << p.first << p.second;
    return 0;
}
```

简答或编程

- 简述函数模板与函数重载的区别与联系。
- 写出一个交换两变量的函数模板 `my_swap`，并在 `main` 中对 `int` 演示。
- 实现类模板 `Stack` 的简化版：内部用数组或 `vector`，提供 `push` / `top`（可不实现完整异常处理）。
- 说明为何模板代码常常放在头文件中。

二、参考答案与解析

1. 答案：B

解析：标准写法是 `template<typename T>`（或 `class T`）。

2. 答案：B

解析：作为类型形参时 `typename` / `class` 等价。

3. 答案：B

解析：模板在编译期为不同类型生成代码。

4. 答案：B

解析：类模板一般要提供模板实参，如 `Box<int>`。

5. 答案：C

解析：实例化与检查发生在编译期。

6. 答案：B

解析：函数模板常可从实参推导。

7. 答案：B

解析：`T` 不能同时是 `int` 与 `double`，需 `add<double>(1, 2.5)` 或统一类型。

8. 答案：A

解析：类外定义需保留模板头并写 `Box<T>::...`。

9. 答案：B

解析：模板有类型检查，优于文本替换的宏。

10. 答案: C

解析: 模板不替代头文件包含机制。

11. 答案: template

解析: 模板声明以 `template` 开始。

12. 答案: class

解析: `template<class T>` 合法且常见。

13. 答案: 实参 (类型实参)

解析: `int` 替换形参 `T`。

14. 答案: 编译

解析: 也称静态多态/编译期多态。

15. 答案: `<int>`

解析: `maximum<int>(3, 4)` 显式指定。

16. 答案:

7

解析: `maximum(3,7)` 返回较大值 7。

17. 答案:

42

解析: `Box<int>` 存取 42。

18. 答案:

16 2.25

解析: `4*4=16`, `1.5*1.5=2.25`。

19. 答案:

A,B

解析: 显式指定 `T` 为 `string` 后输出。

20. 答案:

xy

解析：输出两个 `char` 成员。

21. 答案要点：

重载是多套函数按参数列表选择；模板是一套通用定义，由编译器按类型生成。二者可结合（特化、重载模板等）。

22. 参考代码：

```
#include <iostream>
template<typename T>
void my_swap(T& a, T& b) {
    T t = a; a = b; b = t;
}
int main() {
    int x = 1, y = 2;
    my_swap(x, y);
    std::cout << x << " " << y << "\n";
    return 0;
}
```

23. 参考代码：

```
#include <vector>
template<typename T>
class Stack {
    std::vector<T> data_;
public:
    void push(const T& x) { data_.push_back(x); }
    T& top() { return data_.back(); }
    bool empty() const { return data_.empty(); }
};
```

24. 答案要点：

模板实例化需要完整定义可见；分离到 `.cpp` 时若未显式实例化，其他翻译单元可能链接失败，故常把定义放头文件。

三、本章知识点总结

要点	速记
函数模板	<code>template<typename T> T f(T)</code>
类模板	<code>template<typename T> class C</code>
typename/class	类型形参等价
实例化	编译期生成具体类型版本
推导	函数模板常可自动推导
放置	定义常放头文件

第 11 章 · STL 容器

本章知识点速览

- `vector`：动态数组，`push_back` / `pop_back` / `size` / `clear` / `[]` / `at`
- `map`：键值有序映射，`m[key]=value`，`find`
- `set`：有序不重复集合，`insert` / `count`
- 迭代器：`begin()` / `end()`，`*it` 解引用

一、题目

选择题

- 使用 `std::vector` 需要包含的头文件通常是：
A. `<list>`
B. `<vector>`
C. `<array>`
D. `<deque>`
- `v.push_back(x)` 的作用是：
A. 在头部插入
B. 在尾部追加元素
C. 删除尾部
D. 排序
- 关于 `vector` 的 `operator[]` 与 `at()`，正确的是：
A. 二者越界都一定抛异常
B. `at()` 越界抛异常，`[]` 越界不检查（未定义行为风险）
C. `[]` 更安全
D. `at()` 不能用于 `vector`
- `std::map` 中键的特点通常是：
A. 允许大量重复键且无序
B. 键唯一，并按键有序（默认 `less`）
C. 只能用 `int` 作键
D. 不能用 `[]` 赋值

5. `m.find(key)` 未找到时, 返回值通常是:
- A. `nullptr`
 - B. `m.end()`
 - C. `-1`
 - D. `0`
6. `std::set` 的典型性质是:
- A. 元素可重复且保持插入顺序
 - B. 元素唯一且有序
 - C. 只能存指针
 - D. 随机访问复杂度 $O(1)$
7. `s.count(x)` 对 `set` 而言, 返回值常见含义是:
- A. 元素下标
 - B. 0 或 1 (是否存在)
 - C. 内存字节数
 - D. 迭代器
8. 下列遍历 `vector<int> v` 的写法, 正确的是:
- A. `for (auto it = v.begin(); it != v.end(); ++it)`
 - B. `for (auto it = v.begin(); it == v.end(); ++it)`
 - C. `for (it in v)`
 - D. `foreach v as it`
9. `v.clear()` 之后:
- A. `size()` 为 0
 - B. 对象本身被销毁, 不能再使用 `v`
 - C. 容量一定变为 0
 - D. 等价于 `delete v`
10. 关于迭代器失效, 下列说法较合理的是:
- A. 任何操作都不会使迭代器失效
 - B. `vector` 在扩容/插入等操作后, 旧迭代器可能失效
 - C. `map` 插入后所有迭代器必失效
 - D. 迭代器失效只会在 `set` 发生

填空题

- 11. `vector` 尾部删除元素用 __。
- 12. `vector` 当前元素个数用成员函数 __ 获得。
- 13. `map` 通过键访问并可能插入的下标运算符是 __。
- 14. `set` 插入元素用 __。
- 15. 迭代器解引用运算符是 __。

读程序写结果

16. 写出程序输出:

```

#include <iostream>
#include <vector>
int main() {
    std::vector<int> v;
    v.push_back(1);
    v.push_back(2);
    v.push_back(3);
    std::cout << v.size() << " " << v[1];
    return 0;
}

```

17. 写出程序输出:

```

#include <iostream>
#include <map>
#include <string>
int main() {
    std::map<std::string, int> m;
    m["a"] = 1;
    m["b"] = 2;
    std::cout << m["a"] + m["b"];
    return 0;
}

```

18. 写出程序输出:

```

#include <iostream>
#include <set>
int main() {
    std::set<int> s;
    s.insert(3);
    s.insert(1);
    s.insert(3);
    std::cout << s.size() << " " << s.count(3);
    return 0;
}

```

19. 写出程序输出:

```

#include <iostream>
#include <vector>
int main() {
    std::vector<int> v{10, 20, 30};
    for (auto it = v.begin(); it != v.end(); ++it)
        std::cout << *it;
    return 0;
}

```

20. 写出程序输出:

```
#include <iostream>
#include <map>
int main() {
    std::map<int, int> m;
    m[1] = 100;
    auto it = m.find(2);
    std::cout << (it == m.end() ? "no" : "yes");
    return 0;
}
```

简答或编程

21. 比较 `vector`、`map`、`set` 各自最适合的使用场景（各一句）。
22. 说明 `m[key]` 与 `m.find(key)` 在“键不存在”时行为差异。
23. 编写程序：读入若干整数存入 `vector`，输出元素个数与第一个、最后一个元素（假设非空）。
24. 用 `set` 对输入的一组整数去重并排序后输出（从小到大）。

二、参考答案与解析

1. 答案：B

解析：`vector` 头文件是 `<vector>`。

2. 答案：B

解析：`push_back` 在尾部追加。

3. 答案：B

解析：`at` 做边界检查；`[]` 不检查。

4. 答案：B

解析：`map` 键唯一且有序。

5. 答案：B

解析：未找到返回尾后迭代器 `end()`。

6. 答案：B

解析：`set` 存唯一有序元素。

7. 答案：B

解析：对 `set`，`count` 为 0/1。

8. 答案：A

解析：标准迭代器循环写法。

9. 答案：A

解析：`clear` 清空元素，`size()` 变 0；对象仍可用。

10. 答案：B

解析：`vector` 重分配会使指向旧存储的迭代器失效。

11. 答案: `pop_back()`

解析: 尾删。

12. 答案: `size()`

解析: 返回元素个数。

13. 答案: `[]`

解析: `m[key]` 访问/插入。

14. 答案: `insert`

解析: `s.insert(x)`。

15. 答案: `*`

解析: `*it` 得到元素。

16. 答案:

3 2

解析: 三个元素, 下标 1 为 2。

17. 答案:

3

解析: $1+2=3$ 。

18. 答案:

2 1

解析: 重复插入无效, 大小为 2; ``count(3)==1``。

19. 答案:

102030

解析: 依次输出三元素无空格。

20. 答案:

no

解析: 键 2 不存在, ``find`` 得 ``end()``。

21. 答案要点:

`vector`: 动态顺序表/随机访问; `map`: 按键查找关联数据; `set`: 去重与有序集合查询。

22. 答案要点:

`m[key]` 不存在时会插入默认值元素; `find` 不插入, 未找到返回 `end()`。

23. 参考代码:

```
#include <iostream>
#include <vector>
int main() {
    std::vector<int> v;
    int x;
    while (std::cin >> x) v.push_back(x);
    if (!v.empty())
        std::cout << v.size() << " " << v.front() << " " << v.back() << "\n";
    return 0;
}
```

24. 参考代码:

```
#include <iostream>
#include <set>
int main() {
    std::set<int> s;
    int x;
    while (std::cin >> x) s.insert(x);
    for (int v : s) std::cout << v << " ";
    return 0;
}
```

三、本章知识点总结

要点	速记
vector	动态数组, 尾插删
map	键值映射, 键唯一有序
set	唯一有序集合
find	未找到 → <code>end()</code>
<code>[]</code> vs <code>at</code>	<code>at</code> 更安全
迭代器	<code>begin</code> / <code>end</code> , 注意失效

本章知识点速览

- 头文件: `#include <algorithm>`
- 常用: `sort`、`find`、`count`、`reverse`、`for_each`
- 算法多基于迭代器区间 `[first, last)`
- 与容器配合: 传入 `v.begin()`、`v.end()`

一、题目

选择题

1. 使用 `std::sort` 通常需要包含:
 - A. `<vector>`
 - B. `<algorithm>`
 - C. `<numeric>`
 - D. `<iterator>` 即可, 无需 `algorithm`
2. `sort(v.begin(), v.end())` 的默认效果是:
 - A. 降序排序
 - B. 升序排序
 - C. 去重
 - D. 只排序前半
3. `find(v.begin(), v.end(), x)` 未找到时返回:
 - A. `-1`
 - B. `nullptr`
 - C. `v.end()`
 - D. `0`
4. `count(first, last, value)` 的作用是:
 - A. 删除所有 `value`
 - B. 统计区间中等于 `value` 的个数
 - C. 返回第一个 `value` 的下标
 - D. 排序
5. `reverse(v.begin(), v.end())` 会:
 - A. 对元素排序
 - B. 反转区间内元素顺序
 - C. 清空容器
 - D. 只反转拷贝, 不改原容器
6. 关于 STL 算法区间, 正确的是:
 - A. 一般是闭区间 `[first, last]`
 - B. 一般是左闭右开 `[first, last)`
 - C. 必须是数组, 不能是 `vector`
 - D. `last` 必须指向最后一个元素
7. `for_each` 的典型用途是:
 - A. 对区间每个元素执行函数/函数对象
 - B. 查找最大值
 - C. 合并两个容器
 - D. 分配内存
8. 若要对 `vector<int>` 降序排序, 常用做法是:
 - A. `sort(v.begin(), v.end(), std::greater<int>())`
 - B. `sort(v.end(), v.begin())`

C. `find(v.begin(), v.end())`

D. `count(v.begin(), v.end())`

9. 下列说法错误的是：

A. 算法通过迭代器与容器解耦

B. `sort` 要求随机访问迭代器（如 `vector/deque`）

C. `list` 可直接对 `std::sort(l.begin(), l.end())` 保证高效正确（与 `vector` 相同）

D. `find` 可用于多种容器的迭代器区间

10. 已排序区间可用哪个算法做二分查找（标准库）？

A. `find` 只能

B. `binary_search` / `lower_bound` 等

C. `push_back`

D. `make_shared`

填空题

11. STL 通用算法多在头文件 __ 中。

12. 对整段 `vector` 升序排序： `sort(v.____, v.____)`。

13. `find` 成功时返回指向该元素的 __。

14. `count` 的返回值类型通常可当作 __（数量）。

15. `reverse` 会 __ 修改区间内元素次序。

读程序写结果

16. 写出程序输出：

```
#include <iostream>
#include <vector>
#include <algorithm>
int main() {
    std::vector<int> v{3, 1, 2};
    std::sort(v.begin(), v.end());
    for (int x : v) std::cout << x;
    return 0;
}
```

17. 写出程序输出：

```
#include <iostream>
#include <vector>
#include <algorithm>
int main() {
    std::vector<int> v{1, 2, 3, 2, 2};
    std::cout << std::count(v.begin(), v.end(), 2);
    return 0;
}
```

18. 写出程序输出：

```
#include <iostream>
#include <vector>
#include <algorithm>
int main() {
    std::vector<int> v{1, 2, 3};
    auto it = std::find(v.begin(), v.end(), 2);
    std::cout << (it != v.end() ? *it : -1);
    return 0;
}
```

19. 写出程序输出:

```
#include <iostream>
#include <vector>
#include <algorithm>
int main() {
    std::vector<int> v{1, 2, 3, 4};
    std::reverse(v.begin(), v.end());
    for (int x : v) std::cout << x;
    return 0;
}
```

20. 写出程序输出:

```
#include <iostream>
#include <vector>
#include <algorithm>
int main() {
    std::vector<int> v{5, 4, 3};
    std::sort(v.begin(), v.end());
    std::cout << (std::binary_search(v.begin(), v.end(), 4) ? "Y" : "N");
    return 0;
}
```

简答或编程

21. 说明为何算法使用迭代器而不直接依赖具体容器类型。
22. 写出用 `sort` + 自定义比较 (或 `greater`) 将 `{5,1,4}` 排成降序并输出的程序。
23. 用 `find` 判断 `vector` 中是否存在某值, 存在输出下标, 否则输出 `-1`。
24. 简述 `for_each` 与范围 `for` (`for (auto x : v)`) 各自适用场景。

二、参考答案与解析

1. 答案: B

解析: `sort` 等在 `<algorithm>`。

2. 答案: B

解析: 默认升序 (`operator<`)。

3. 答案: C

解析: 未找到返回 `last`, 对容器即 `end()`。

4. 答案: B

解析: `count` 统计相等元素个数。

5. 答案: B

解析: 原地反转顺序。

6. 答案: B

解析: STL 惯例半开区间。

7. 答案: A

解析: 对每个元素应用可调用对象。

8. 答案: A

解析: 传入比较器 `greater` 实现降序。

9. 答案: C

解析: `std::sort` 需要随机访问迭代器; `list` 应使用成员 `sort`。

10. 答案: B

解析: 有序区间用 `binary_search` / `lower_bound` 等。

11. 答案: `<algorithm>`

解析: 算法头文件。

12. 答案: `begin(); end()`

解析: 传入起止迭代器。

13. 答案: 迭代器

解析: 指向匹配元素。

14. 答案: 整数/大小 (`difference_type` / `size_t` 等)

解析: 表示个数。

15. 答案: 原地 (就地)

解析: 直接改原序列。

16. 答案:

123

解析: 排序后 1,2,3。

17. 答案:

3

解析：值为 2 的元素有 3 个。

18. 答案：

2

解析：找到元素 2。

19. 答案：

4321

解析：反转后输出。

20. 答案：

Y

解析：排序后二分查找 4 成功。

21. 答案要点：

迭代器抽象了遍历方式，使同一算法可用于数组、`vector` 等多种序列，实现算法与容器解耦。

22. 参考代码：

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <functional>
int main() {
    std::vector<int> v{5, 1, 4};
    std::sort(v.begin(), v.end(), std::greater<int>());
    for (int x : v) std::cout << x << " ";
    return 0;
}
```

23. 参考代码：

```
#include <iostream>
#include <vector>
#include <algorithm>
int main() {
    std::vector<int> v{10, 20, 30};
    int x = 20;
    auto it = std::find(v.begin(), v.end(), x);
    if (it == v.end()) std::cout << -1;
    else std::cout << (it - v.begin());
    return 0;
}
```

24. 答案要点:

`for_each` 便于传入函数对象/泛型算法组合；范围 `for` 语法更简洁，适合简单遍历。复杂算法链或泛型代码中 `for_each` 更常见。

三、本章知识点总结

要点	速记
头文件	<code><algorithm></code>
区间	<code>[first, last)</code>
sort	默认升序
find/count	查找 / 计数
reverse	原地反转
for_each	对每个元素执行操作

第 13 章 · 文件流

本章知识点速览

- 头文件: `#include <fstream>`
- `ifstream` 读、`ofstream` 写、`fstream` 读写
- 写: `ofs << ...`; 读: `ifs >>` 或 `getline`
- 用完可 `close()`; 对象析构也会关闭 (RAII)

一、题目

选择题

1. C++ 文件流相关类主要在哪个头文件?
- A. `<iostream>`
 - B. `<fstream>`

- C. `<cstdio>` 只能
- D. `<file>`
- 2. 专用于读文件的流类型是:
 - A. `ofstream`
 - B. `ifstream`
 - C. `stringstream`
 - D. `ostream` 仅此一个
- 3. 专用于写文件的流类型是:
 - A. `ifstream`
 - B. `ofstream`
 - C. `istream`
 - D. `cin`
- 4. 打开文件写文本的常见写法是:
 - A. `std::ofstream ofs("out.txt");`
 - B. `std::ifstream ofs("out.txt");`
 - C. `fopen_write("out.txt")`
 - D. `std::cout.open("out.txt")`
- 5. 判断文件是否成功打开, 常用:
 - A. 只能看返回值 `int`
 - B. `if (!ifs) / ifs.is_open()`
 - C. `ifs == 0`
 - D. 无法判断
- 6. 按行读取文本文件常用:
 - A. `std::getline(ifs, line)`
 - B. `ifs.getline_all()`
 - C. `gets(ifs)`
 - D. `scanf_line`
- 7. 关于 `close()`, 正确的是:
 - A. 必须手动调用, 否则文件永不能关闭
 - B. 可手动调用; 流对象析构时通常也会关闭
 - C. `close` 只对 `cin` 有效
 - D. 关闭后还能继续写入同一流对象无需重开
- 8. `fstream` 的特点是:
 - A. 只能读
 - B. 只能写
 - C. 可读可写 (按打开模式)
 - D. 只能用于二进制且无法文本
- 9. 向文件输出与向 `cout` 输出在语法上:
 - A. 完全不同, 文件必须用 `printf`
 - B. 都可用 `<<` 运算符
 - C. 文件只能用 `>>`
 - D. 不能输出字符串
- 10. 若 `ifstream ifs("no_such.txt");` 文件不存在, 通常:
 - A. 程序立刻崩溃且无法检测
 - B. 流处于失败状态, 可用布尔检测
 - C. 自动创建该文件并读到数据
 - D. 一定抛出必须捕获的异常 (默认)

填空题

11. 读文件类名是 __，写文件类名是 __。
12. 文件流头文件是 __。
13. 使用 << 写入文件类似于写入 __。
14. 逐行读取可用函数 __。
15. 文件流对象离开作用域时，依据 __ 原则自动关闭文件。

读程序写结果

16. 写出程序输出（假设运行前不存在 t.txt，关注控制台输出）：

```
#include <iostream>
#include <fstream>
#include <string>
int main() {
    {
        std::ofstream ofs("t.txt");
        ofs << "Hi";
    }
    std::ifstream ifs("t.txt");
    std::string s;
    ifs >> s;
    std::cout << s;
    return 0;
}
```

17. 写出程序输出：

```
#include <iostream>
#include <fstream>
#include <string>
int main() {
    std::ofstream ofs("a.txt");
    ofs << "A\nB\n";
    ofs.close();
    std::ifstream ifs("a.txt");
    std::string line;
    std::getline(ifs, line);
    std::cout << line;
    return 0;
}
```

18. 写出程序输出：

```
#include <iostream>
#include <fstream>
int main() {
    std::ifstream ifs("definitely_missing_xyz_123.txt");
    std::cout << (ifs ? "open" : "fail");
    return 0;
}
```

19. 写出程序输出:

```
#include <iostream>
#include <sstream>
#include <string>
int main() {
    std::istringstream iss("10 20");
    int a, b;
    iss >> a >> b;
    std::cout << a + b;
    return 0;
}
```

20. 写出程序输出:

```
#include <iostream>
#include <fstream>
int main() {
    std::ofstream ofs("n.txt");
    ofs << 1 << 2 << 3;
    ofs.close();
    std::ifstream ifs("n.txt");
    int x;
    ifs >> x;
    std::cout << x;
    return 0;
}
```

简答或编程

21. 对比 `ifstream`、`ofstream`、`fstream` 的用途。
22. 写出将字符串 "Hello, file!" 写入 `out.txt` 的完整小程序。
23. 写出从 `in.txt` 逐行读取并原样输出到控制台的程序。
24. 说明为何文件流也符合 RAII，这对异常安全有何好处。

二、参考答案与解析

1. 答案: B

解析: 文件流在 `<fstream>`。

2. 答案: B

解析: `ifstream` 输入文件流。

3. 答案: B

解析: `ofstream` 输出文件流。

4. 答案: A

解析: 构造 `ofstream` 并指定文件名即可打开写入。

5. 答案: B

解析: 流可转换为布尔上下文, 或用 `is_open()`。

6. 答案: A

解析: `std::getline` 读一行到 `string`。

7. 答案: B

解析: 可显式 `close`; 析构也会关闭。

8. 答案: C

解析: `fstream` 支持读写 (取决于 mode)。

9. 答案: B

解析: 同样使用插入运算符 `<<`。

10. 答案: B

解析: 默认不抛异常, 流置失败态。

11. 答案: `ifstream; ofstream`

解析: 输入/输出文件流。

12. 答案: `<fstream>`

解析: 文件流头文件。

13. 答案: `cout` (标准输出)

解析: 接口一致。

14. 答案: `std::getline`

解析: 按行读取。

15. 答案: `RAII`

解析: 资源随对象生命周期管理。

16. 答案:

Hi

解析: 写入后读回并输出。

17. 答案:

A

解析：`getline` 只读第一行。

18. 答案：

fail

解析：文件不存在，流失败。

19. 答案：

30

解析：字符串流读入 10 与 20 相加（说明流提取语法；与文件流类似）。

20. 答案：

123

解析：无分隔写入 `123`，一次 `>>` 读得整数 123。

21. 答案要点：

`ifstream` 读文件；`ofstream` 写文件；`fstream` 同一流对象读写（配合打开模式）。

22. 参考代码：

```
#include <fstream>
int main() {
    std::ofstream ofs("out.txt");
    ofs << "Hello, file!\n";
    return 0;
}
```

23. 参考代码：

```
#include <iostream>
#include <fstream>
#include <string>
int main() {
    std::ifstream ifs("in.txt");
    std::string line;
    while (std::getline(ifs, line))
        std::cout << line << "\n";
    return 0;
}
```


24. 答案要点：

打开文件作为对象构造的一部分，析构自动关闭；即使中途抛异常也能关闭文件，减少泄漏与遗忘 `close` 的风险。

三、本章知识点总结

要点	速记
头文件	<code><fstream></code>
读/写	<code>ifstream</code> / <code>ofstream</code>
读写	<code>fstream</code>
写入	<code>ofs << data</code>
读行	<code>getline(ifs, line)</code>
关闭	<code>close()</code> 或析构 (RAII)

第 14 章 · 智能指针与常用头文件

本章知识点速览

- `unique_ptr`：独占所有权，常用 `make_unique`
- `shared_ptr`：共享所有权，引用计数，常用 `make_shared`
- `weak_ptr`：弱引用，不增加引用计数，可破坏
- RAII：构造获资源、析构释资源；头文件：`iostream` / `string` / `vector` / `map` / `set` / `algorithm` / `memory` / `fstream` / `array`

一、题目

选择题

- 智能指针相关设施主要在头文件：
A. `<pointer>`
B. `<memory>`
C. `<cstdlib>`
D. `<new>` 即可含全部智能指针
- `unique_ptr` 的核心语义是：
A. 多个指针共享同一对象且一样安全无限制
B. 独占所有权，不可拷贝（可移动）
C. 不拥有对象
D. 只能管理数组
- 创建 `unique_ptr<int>` 的推荐写法（C++14 起）是：
A. `std::unique_ptr<int> p(new int(1));` 唯一合法
B. `auto p = std::make_unique<int>(1);`

- C. `std::shared_ptr<int> p = ...` 即 `unique`
 - D. `int* p = make_unique`
4. `shared_ptr` 通过什么管理共享对象寿命?
- A. 垃圾回收线程
 - B. 引用计数
 - C. 手动 `free`
 - D. 操作系统定时器
5. `weak_ptr` 的主要用途之一是:
- A. 替代所有 `unique_ptr`
 - B. 观察 `shared_ptr` 管理的对象而不增加引用计数 (可打破环)
 - C. 加快 `new` 速度
 - D. 管理文件句柄专用
6. 关于 RAII, 正确的是:
- A. 资源在构造时获取, 析构时释放
 - B. 资源必须在 `main` 开头全部申请
 - C. 与智能指针无关
 - D. 只适用于 Java
7. `<algorithm>` 中常见的是:
- A. `std::cout`
 - B. `std::sort`、`std::find`
 - C. `std::unique_ptr`
 - D. `std::ifstream`
8. `<vector>` 提供的是:
- A. 动态数组容器 `std::vector`
 - B. 智能指针
 - C. 文件流
 - D. 排序算法本体 (定义处)
9. 下列配对正确的是:
- A. `<fstream>` → `std::string`
 - B. `<map>` → `std::map`
 - C. `<memory>` → `std::cin`
 - D. `<set>` → `std::sort`
10. 为何更推荐 `make_unique` / `make_shared` 而不是直接 `new` ?
- A. 它们更慢所以更安全
 - B. 异常安全更好、写法更简洁, 并封装分配细节
 - C. 可以避免包含任何头文件
 - D. 能让指针变成栈对象本身

填空题

- 11. 独占智能指针类型名是 __。
- 12. 共享智能指针类型名是 __。
- 13. 弱引用智能指针类型名是 __。
- 14. `shared_ptr` 头文件是 __。
- 15. RAII 全称常译为: 资源获取即 __。

读程序写结果

- 16. 写出程序输出:

```
#include <iostream>
#include <memory>
int main() {
    auto p = std::make_unique<int>(10);
    std::cout << *p;
    return 0;
}
```

17. 写出程序输出:

```
#include <iostream>
#include <memory>
int main() {
    auto a = std::make_shared<int>(5);
    auto b = a;
    std::cout << a.use_count();
    return 0;
}
```

18. 写出程序输出:

```
#include <iostream>
#include <memory>
int main() {
    std::shared_ptr<int> sp = std::make_shared<int>(7);
    std::weak_ptr<int> wp = sp;
    std::cout << sp.use_count() << " ";
    if (auto locked = wp.lock())
        std::cout << *locked;
    return 0;
}
```

19. 写出程序输出:

```
#include <iostream>
#include <memory>
struct Foo {
    Foo() { std::cout << "C"; }
    ~Foo() { std::cout << "D"; }
};
int main() {
    {
        auto p = std::make_unique<Foo>();
    }
    std::cout << "E";
    return 0;
}
```

20. 写出程序输出:

```
#include <iostream>
#include <vector>
#include <algorithm>
int main() {
    std::vector<int> v{2, 1};
    std::sort(v.begin(), v.end());
    std::cout << v[0] << v[1];
    return 0;
}
```

简答或编程

21. 对比 `unique_ptr` 与 `shared_ptr` 的所有权模型与选用原则。
22. 解释循环引用问题，以及 `weak_ptr` 如何帮助解决。
23. 用 `unique_ptr` 管理一个动态 `int` 数组或单个对象，并在函数结束前打印值（展示无需手动 `delete`）。
24. 列出至少 6 个常用标准头文件及其主要提供的内容。

二、参考答案与解析

1. 答案：B

解析：智能指针在 `<memory>`。

2. 答案：B

解析：`unique_ptr` 独占，不可拷贝可移动。

3. 答案：B

解析：推荐 `make_unique`。

4. 答案：B

解析：共享所有权靠引用计数。

5. 答案：B

解析：弱引用不增加 `use_count`，常用于破坏。

6. 答案：A

解析：RAII 绑定资源与对象生命周期。

7. 答案：B

解析：算法如 `sort` / `find`。

8. 答案：A

解析：`vector` 容器。

9. 答案：B

解析：`<map>` 对应 `std::map`。

10. 答案：B

解析：更安全简洁，减少裸 `new` 异常问题。

11. 答案: **unique_ptr**

解析: 独占智能指针。

12. 答案: **shared_ptr**

解析: 共享智能指针。

13. 答案: **weak_ptr**

解析: 弱引用。

14. 答案: `<memory>`

解析: 智能指针头文件。

15. 答案: **初始化**

解析: Resource Acquisition Is Initialization。

16. 答案:

10

解析: 解引用 `unique\_ptr` 得 10。

17. 答案:

2

解析: 两个 `shared\_ptr` 共享, 引用计数为 2。

18. 答案:

1 7

解析: `weak\_ptr` 不增加引用计数; 打印 `use\_count` 时仍为 1; `lock()` 后可读到 7。

19. 答案:

CDE

解析: 构造输出 `C`, 离开作用域析构输出 `D`, 再输出 `E`。

20. 答案:

12

解析: 排序后为 1、2。

21. 答案要点:

`unique_ptr` 独占，默认不可拷贝，适合单一所有者；`shared_ptr` 共享并引用计数，适合多处共享同一资源。能独占则优先 `unique_ptr`。

22. 答案要点:

两个 `shared_ptr` 互相持有会导致引用计数无法归零；将其中一侧改为 `weak_ptr`，不增加引用，环可断开，对象得以释放。

23. 参考代码:

```
#include <iostream>
#include <memory>
int main() {
    auto p = std::make_unique<int>(42);
    std::cout << *p << "\n";
    return 0; // 自动 delete
}
```

24. 答案要点:

`<iostream>`: cin/cout; `<string>`: string; `<vector>`: vector; `<map>`/`<set>`: 关联容器;
`<algorithm>`: sort/find 等; `<memory>`: 智能指针; `<fstream>`: 文件流; `<array>`: array。

三、本章知识点总结

要点	速记
<code>unique_ptr</code>	独占, <code>make_unique</code>
<code>shared_ptr</code>	共享, 引用计数, <code>make_shared</code>
<code>weak_ptr</code>	弱引用, 破坏
RAII	构造获取, 析构释放
<code><memory></code>	智能指针
常用头	<code>iostream/string/vector/map/set/algorithm/fstream/array</code>